

ParaKplex: A parallel local search algorithm for the maximum K-Plex problem

Jieyu Wu^{a,b,†}, Rui Sun^{c,†}, Yiyuan Wang^{a,b,*}, Minghao Yin^{a,b}

^a School of Information Science and Technology, Northeast Normal University, Changchun, 130117, China

^b Key Laboratory of Applied Statistics of MOE, Northeast Normal University, Changchun, 130117, China

^c College of Information Science and Engineering, Northeastern University, Shenyang, 110819, China

ARTICLE INFO

Keywords:

Optimization

Maximum k -plex problem

Local search

Group-based initialization

Primary-secondary search

Large graph

ABSTRACT

The maximum k -plex problem (MKPP) is a generalized relaxation of the maximum clique problem with numerous practical applications. Recent advancements have introduced efficient algorithms for solving the MKPP, particularly local search algorithms that exhibit excellent performance. However, most state-of-the-art approaches remain sequential, with limited research on parallelization. In this work, we propose a parallel local search algorithm named ParaKplex to solve the MKPP. To parallelize the algorithm, we design a group-based initialization method for generating initial solutions across threads and a primary-secondary search method for integrating global and local search processes. Moreover, our algorithm adopts three novel sequential strategies, including a new configuration checking strategy to relax the forbidden condition of vertices, a vertex selection strategy to identify high-quality vertices, and a perturbation strategy to help the algorithm escape local optima. Comprehensive experiments demonstrate that ParaKplex significantly outperforms existing state-of-the-art algorithms. Additionally, the experimental analysis highlights the scalability of the parallelization approach and the effectiveness of the proposed strategies.

1. Introduction

Cohesive subgroups are a common concept in social network analysis and form the basis of many sociological theories [1]. The earliest mathematical model of a cohesive subgroup was the clique. Formally, a clique model is a subset of vertices of an undirected graph such that every two distinct vertices in the subset are adjacent. Clique is an ideal model, where members have no strangers to communicate directly with each other, and it is impossible to destroy the subgroup by removing members. However, the constraints of clique model are too restrictive in practice [2,3]. In fact, a real-life cohesive subgroup could include some strangers. In order to deal with real-world scenarios, various efforts have been made to relax this requirement [4], and there are various clique relaxation problems, such as k -plex [5], k -defective-clique [6], and k -quasi-clique [7]. In our study, we focus on proposing an efficient algorithm to deal with the k -plex problem.

Given an undirected graph $G = (V, E)$ and a positive integer k , a k -plex S is a subset of vertices such that each vertex in S is at least adjacent to $|S| - k$ other vertices. The aim of the maximum k -plex problem (MKPP) is to find a k -plex with the most vertices.

* Corresponding author.

E-mail addresses: wujy939@nenu.edu.cn (J. Wu), sunr@ise.neu.edu.cn (R. Sun), yiyuanwangjlu@126.com (Y. Wang), ymh@nenu.edu.cn (M. Yin).

† These authors contributed equally to this work.

Because of its relaxed constraints, the k -plex model is particularly well suited for the analysis of graph instances that arise from real-world applications. Nowadays, the MKPP has been applied to various fields including data mining [8], social network analysis [4] and scalable community detection [9]. Besides, based on the flexibility of k -plex, appropriately increasing the value of k helps to discover larger tight community structures in social networks. This flexibility is suited for handling weak ties in interpersonal relationships, where the connections between members are not so close and may even involve strangers [10]. A larger k value allows for more weak ties among community members. In social networks, such weak ties can often provide important clues in criminal detection, so as to dig out potential criminal social networks [11]. Therefore, in this study, in addition to solving the commonly used k values in the previous works [12–14], we also increase the k to a large value to find larger cohesive subgraphs in the undirected graph.

In theory, the MKPP is proven to be an NP-hard problem [15]. Under the assumption that $P \neq NP$, no polynomial-time algorithm exists for such problems. Thus, it is not an easy task to solve the MKPP. Many researchers have developed various methods to solve the MKPP, mainly including exact and heuristic algorithms. In recent years, several exact algorithms have been proposed for dealing with the MKPP, the most widely used is the branch-and-bound (BnB) algorithm [16–22]. For example, to solve large sparse graphs, Chang et al. [23] iteratively extract small dense subgraphs from the graph and then solve each extracted subgraph using a BnB search. Jiang et al. [24] proposed a new BnB algorithm called KpLeX, which utilizes an upper bound determined by partitioning the candidate vertex set in relation to the current growing solution. After that, the authors further improved this upper bound to enable tighter pruning, which can be effective in reducing the search space [25]. In the same year, Wang et al. [26] proposed the KpLeX-Gap algorithm, which calculates the gap between the graph's degeneracy bound and the maximum k -plex size to determine if certain subgraphs can contain a k -plex. Afterwards, in 2024, Chang et al. proposed a two-stage algorithm named kPlexT [12]. In the first stage, kPlexT incorporates reduction rules and branching rules to reduce the search space; in the second stage, it combines an initialization method and the bounding technique to optimize the performance. Subsequently, the kPEX algorithm proposed by Gao et al. [13] first partitions a branch into two parts, then alternately conducts the reduction process and the bounding process within each branch, effectively improving the algorithm's performance. A BnB algorithm called RelaxPUB-Maplex [27], combines the complementary strengths of graph coloring and graph partitioning techniques to design a novel upper bound method, which can effectively improve the solution performance. Recently, Ahmad et al. [28] have proposed a parallel BnB algorithm called U-MKP, which integrates various pruning techniques from previous BnB algorithms. Among those algorithms, kPlexT [12], kPEX [13] and RelaxPUB-Maplex [27] are the three most recent and most efficient algorithms. U-MKP is the state-of-the-art parallel exact algorithm for the MKPP. However, both algorithms exhibit certain limitations. In particular, as the value of k increases, their performance deteriorates, and solving some of the more difficult instances within a reasonable time becomes increasingly challenging.

Researchers have also developed heuristics to quickly provide a solution for the MKPP. The representative heuristic algorithm for the MKPP mainly uses stochastic local search [5,29–34]. Recently, a GPU-based local search algorithm called GUPLS [35] was proposed to solve the weighted and unweighted MKPP, and two new neighborhood structures were designed to facilitate efficient parallel search. In the same year, DCCplex [5] was introduced. It employs a dynamic-threshold configuration checking strategy to avoid repeatedly revisiting the same solutions during the search, along with an incremental updating scheme based on the number of non-neighbors in the current candidate solution and saturated vertex set to efficiently identify feasible neighborhoods. After that, Pullan [34] proposed a local search algorithm called KLS, which utilizes data structures suitable for graph analysis and incorporates mechanisms to enhance search diversity. Recently, Sun et al. [36] proposed a general group-driven initialization (GDI) method and constructed a good initial solution by considering the generated group information. By integrating the GDI into the MKPP algorithms FD-TS [37] and DCCplex [5], they proposed new algorithms, FD-TS_GDI and DCCplex_GDI. Compared to the original algorithms, these GDI-based algorithms significantly enhanced the solution quality. As far as we know, FD-TS_GDI and DCCplex_GDI are the best-performing heuristic algorithms.

To further improve the performance of solving the MKPP, in this study, we propose a parallel local search algorithm called ParaKplex. The primary contributions of our work can be summarized in five points, with the first two specifically focused on algorithm parallelization.

First, we propose a novel group-based initialization method which consists of subgroup generation, subgroup update and group-based initialization procedures. We utilize the information from k -shell decomposition [38] to generate the initial subgroups. Subsequently, the subgroups are updated based on the search information provided by the worker threads. The dynamically updated subgroups are used to generate initial solutions for worker threads when the search is restarted.

Second, we propose a primary-secondary search framework to facilitate thread cooperation and balance global and local exploration. The primary search obtains an initial solution from the group-based initialization procedure and then improves the solution by exploring the entire search space. The secondary search guides the algorithm to quickly build a solution in dense regions based on the search information provided by threads. These two searches are executed in parallel rather than sequentially. Once either search completes, the local best solution is used to update the global best solution, reduce the given graph, and update the subgroups. The updated subgroup information and search information are used to facilitate the collaborative search of these two modes.

Third, we introduce a novel variant of the configuration checking (CC) strategy. CC is a widely used strategy for the cycling problem and has been utilized in numerous combinatorial optimization problems [39]. Motivated by the characteristics of the MKPP and the three basic operations (i.e., Add, Swap, and Drop) of local search algorithms for the MKPP, we develop a relaxed bounded configuration checking strategy (RBCC). In our strategy, we relax the restricted strength of configuration checking by incorporating the vertices' frequency and utilize a tabu mechanism [40] to enable more candidate vertices to be selected for addition during the search procedure.

Fourth, we introduce a novel solution information-based vertex selection strategy based on two scoring functions. The first scoring function combines the structure information of the given graph (i.e., degree value) and the current solution, whereas the second scoring function leverages the historical information of the search procedure (i.e., local best solution). It is worth noting that these scoring functions assess the gain of adding a vertex from distinct perspectives.

Fifth, we propose a core-based perturbation strategy aimed at diversifying the search direction. Our perturbation strategy defines a solution core, which is obtained by iteratively removing saturated vertices from the solution until no saturated vertices remain. Building on the definition of the solution core, our perturbation strategy extracts a cohesive subset from the candidate solution. It then extends this subset by adding vertices to form a perturbed solution, continuing until the perturbed solution sufficiently differs from the original solution.

By incorporating these ideas, we present an efficient parallel local search algorithm named ParaKplex. To evaluate the effectiveness of ParaKplex, we conducted extensive experiments on benchmark instances for the MKPP that have been widely used in previous literature. The experimental results show that ParaKplex outperforms the state-of-the-art algorithms on all benchmarks, and the proposed strategies contribute significantly to the superior performance of our algorithm.

This study is an improvement on our conference paper [41], which proposes a local search algorithm called Nukplex. Nukplex is a sequential algorithm, while ParaKplex is a parallel algorithm. There are three main differences between them. First, ParaKplex employs a primary-secondary search mode to coordinate the search among worker threads, while Nukplex uses a single search mode. Specifically, ParaKplex adopts the same search mode as Nukplex as its primary search mode and introduces a new search mode to quickly construct solutions in dense regions. Second, the initialization methods used in ParaKplex and Nukplex differ. ParaKplex employs a group-based initialization method, which divides vertices into subgroups based on the search information collected from each worker thread and generates initial solutions accordingly. In contrast, Nukplex uses an initialization approach based on vertex selection frequency. Third, we have enriched the experiments on ParaKplex. ParaKplex incorporates two additional k values (i.e., 25 and 30) to detect larger cohesive subgroups, resulting in a total of nine k values. Moreover, our experimental evaluation includes five additional comparison algorithms: a parallel local search algorithm (GPULS [35]), a parallel exact algorithm (U-MKP [28]), and three state-of-the-art exact algorithms (kPEX [13], KplexT [12] and RelaxPUB-Maplex [27]).

In the next section, we introduce some background knowledge. Section 3 introduces the ParaKplex algorithm, followed by a description of the group-based initialization method in Section 4. Three novel search strategies are presented in Section 5. Section 6 details the primary-secondary search method. Section 7 discusses the differences between Nukplex and ParaKplex. Experimental results are reported in Section 8. Finally, we make conclusions.

2. Preliminaries

Given an undirected graph $G = (V, E)$, V is a vertex set and E is an edge set. The edge density of G is defined as $dens(G) = \frac{2|E|}{|V|(|V|-1)}$. Two vertices are considered neighbors if and only if they both belong to the same edge. For each vertex $v \in V$, the set of its neighbors is defined as $N(v) = \{u \in V \mid (v, u) \in E\}$ and the degree of v is $d_G(v) = |N(v)|$. Given a vertex set $S \subseteq V$, $N(S) = \bigcup_{v \in S} N(v) \setminus S$ is the set of neighbors of S . The induced graph $G[S] = (S, E_S)$ is a subgraph of G whose vertex set is S and whose edge set includes all the edges in E where both endpoints are in S .

Given an undirected graph G and a positive integer k , a subset S of V is a k -plex if and only if $|N(v) \cap S| \geq |S| - k$ for each $v \in S$. The goal of the maximum k -plex problem is to find a k -plex with the largest size.

In the search process, we use S to denote the current feasible candidate solution (i.e., being a k -plex). For a vertex $v \in S$, if $|N(v) \cap S| = |S| - k$, v is called a saturated vertex. Let $C(S)$ denotes the set of all saturated vertices in S . For the MKPP, local search algorithms typically modify S iteratively through Add, Swap, and Drop operations [5,37]. We introduce three candidate sets as follows:

- $AddSet(S) = \{v \in N(S) \mid |N(v) \cap S| > |S| - k, C(S) \setminus N(v) = \emptyset\}$;
- $SwapSet(S) = \{v \in N(S) \mid |N(v) \cap S| \geq |S| - k, |C(S) \setminus N(v)| = 1\} \cup \{v \in N(S) \mid |N(v) \cap S| = |S| - k, |C(S) \setminus N(v)| = \emptyset\}$;
- $DropSet(S) = S$.

The vertices contained in $AddSet(S)$ are adjacent to S with more than $|S| - k$ neighbors and adjacent to all saturated vertices, which can be added directly to S . The Swap operation is used to exchange a pair of vertices of S and $V \setminus S$. The vertices contained in $SwapSet(S)$ can be exchanged only if one of these two conditions is satisfied: 1) v is adjacent to at least $|S| - k$ vertices in S , and v is not adjacent to only one saturated vertex; 2) v is adjacent to exactly $|S| - k$ vertices in S , and the set of v 's neighbors contains all saturated vertices. In the Swap operation, the algorithm adds a vertex $v \in SwapSet(S)$ into S and randomly removes one vertex from S . The vertex is considered to be removed in two cases. In the first case, the unique saturated vertex that is not adjacent to v will be removed; in the second case, a vertex from $S \setminus N(v)$ will be removed. The Drop operation is used to remove a vertex from S . $DropSet(S)$ contains all vertices in S and is considered only when $AddSet(S)$ and $SwapSet(S)$ are empty.

3. A parallel local search algorithm for the MKPP

We propose a parallel local search algorithm named ParaKplex for the MKPP. As illustrated in Fig. 1, the framework of ParaKplex contains a master thread and several worker threads. We introduce the work of these threads as follows.

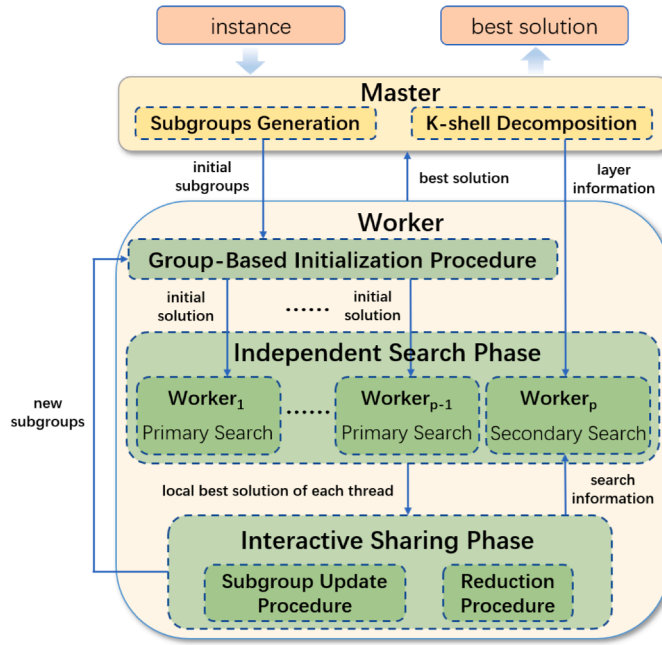


Fig. 1. The parallel framework of the ParaKplex algorithm.

Master thread. The master thread reads the instance and constructs the data structures used during the search. The master thread is equipped with two procedures, including k -shell decomposition and subgroup generation (see Sections 4.1 and 4.2). These two procedures divide the vertices in G into two groups of vertex sets, namely the k -shell layer set $\mathcal{G}^{ks}$ and the subgroup set $\mathcal{G}^{sg}$, respectively. Our algorithm incorporates the layer information derived from k -shell decomposition to generate the subgroups $\mathcal{G}^{sg}$ during the subgroup generation procedure. The subgroup set $\mathcal{G}^{sg}$ is used in the group-based initialization procedure to construct initial solutions. When the time limit is reached, the master thread outputs the best solution found in the threads.

Worker threads. The worker threads operate in two phases, including the independent search phase and the interactive sharing phase. We describe the working details of the worker threads in these two phases as follows.

In the independent search phase, worker threads employ a primary-secondary search method (see Section 6), which consists of two complementary search modes to fully explore the search space. The primary search mode begins by generating initial solutions through the group-based initialization procedure (see Section 4.4), and then iteratively performs local search processes to enhance the current solution. Meanwhile, the secondary search mode leverages subgroup information derived from k -shell decomposition to quickly construct high-quality solutions. Once either search mode completes its process, the local best solution is passed to the interactive sharing phase. In the primary search, the dynamically updated subgroups from the interactive sharing phase guide the group-based initialization procedure to generate higher-quality initial solutions. In the secondary search, the search information from the interactive sharing phase is used to guide the algorithm in constructing solutions within more promising regions. The secondary search mode focuses on dense regions, whereas the primary search mode ensures comprehensive exploration of the entire search space. In our work, our algorithm assigns one worker thread to the secondary search mode and dedicates the remaining threads to the primary search mode.

In the interactive sharing phase, worker threads are responsible for updating five global variables, including the global best solution S_{gb} , the given graph G , the subgroup set $\mathcal{G}^{sg}$, and the weights w^{ks} of layers in $\mathcal{G}^{ks}$ and w^{sg} of subgroups in $\mathcal{G}^{sg}$. Only one thread is permitted to update these variables at a time, and we use read-write locks [42] to protect these global variables. 1) For updating the global best solution S_{gb} and the graph G , a worker thread must acquire a write lock. Once a thread successfully obtains the lock, it updates S_{gb} and G based on its locally found solution. If other threads attempt to update these variables while the lock is held, they do not block but instead continue their local search for 100 additional iterations before re-attempting to acquire the lock. 2) For updating the subgroup set $\mathcal{G}^{sg}$ as well as the weights w^{ks} and w^{sg} , our experiments indicate that the average update time for $\mathcal{G}^{sg}$ is about 0.03 seconds, while updating the weights w^{ks} and w^{sg} takes about 9×10^{-7} seconds. The overall impact of these update operations on the algorithm's efficiency is negligible. Consequently, when a thread acquires write locks for updating these variables, other threads simply wait for the locks to be released before proceeding with their own updates. There are two procedures used in the interactive sharing phase, including the subgroup update procedure (see Section 4.3) and the reduction procedure.

3.1. The details of ParaKplex

In this subsection, we first introduce some key variables used in ParaKplex, followed by a detailed description of the algorithm.

In the search process, we introduce two vertex attributes for the i -th worker thread, $freq_i$ and $step_i$.

- For each vertex $v \in V$, $freq_i(v)$ represents the frequency of v that is added to or removed from the solution in the i -th worker thread. Initially, the value of $freq_i(v)$ is set to 0. Each time the vertex v is either added to or removed from the current solution, its $freq_i(v)$ value is incremented by 1.
- For each vertex $v \in V$, $step_i(v)$ represents the steps that v remains in the current solution in the i -th worker thread. Initially, the value of $step_i(v)$ is initialized to 0. Once v is added to the solution, $step_i(v)$ increases by 1 in each iteration and stops increasing when the vertex is removed from the solution. When v is added to the solution again, the $step_i(v)$ value continues to increase.

In addition, for the i -th worker thread, S_{lb_i} denotes the local best solution, $iter_i$ represents the number of restarts, α_i controls the frequency of subgroup updates, $prefreq_i$ is used to calculate the increment in vertex frequency, and $updfreq_i$ tracks the number of subgroup updates.

The details of the ParaKplex algorithm are presented in [Algorithm 1](#). ParaKplex consists of three phases: the initialization phase (Lines 1–3), the independent search phase (Lines 6–12), and the interactive sharing phase (Lines 13–19). Finally, the algorithm returns the global best solution (Line 20).

In the initialization phase, S_{gb} denotes the global best solution and is initialized to an empty set. The algorithm calls the k -shell decomposition procedure and the subgroup generation procedure to obtain the k -shell layer set $\mathcal{G}^{ks}$ and the initial subgroup set $\mathcal{G}^{sg}$, respectively (Line 1). The variables used in each worker thread are initialized in Lines 2–3.

In the independent search phase, the worker threads execute the primary-secondary search in parallel and return the local best solution (Lines 6–7). After that, $iter_i$ is increased by 1 (Line 8). When a worker thread has executed the search mode T times (where T is a parameter), if the subgroup update has been performed more than $T/2$ times, α_i is halved (Line 10); otherwise, α_i is doubled (Line 11). Subsequently, the value of $updfreq_i$ is reset to 0 (Line 12).

In the interactive sharing phase, the algorithm updates S_{gb} according to the S_{lb_i} found by each worker thread, and then invokes the *Reduce* procedure to reduce the given graph (Line 14). The *Reduce* procedure removes vertices with a degree value less than $|S_{gb}| - (k - 1)$. If G is empty or the remaining vertices in G are fewer than $|S_{gb}|$, it means the algorithm has found the global best solution and returns S_{gb} (Line 15). If the increment in vertex frequency reaches or exceeds $\alpha_i \times |V|$, indicating that the subgroup set needs to be updated, the algorithm calls the *SGU pdate* procedure (see [Section 4.3](#)) (Line 17). The values of $updfreq_i$ and $prefreq_i$ are updated accordingly (Lines 18–19). Furthermore, we summarize the key variables of each procedure in a table in the appendix.

4. A novel group-based initialization method

In this section, we propose a group-based initialization method. To obtain the subgroups, we first describe the procedures for subgroup generation and update, and then propose the initialization method. The core idea of this initialization method is to leverage the local best solutions found by each worker thread to partition dense regions into subgroups, and generate promising initial solutions for the worker threads based on these subgroups.

4.1. Previous k -shell decomposition method

The k -shell decomposition is a community detection method in complex network analysis [38]. In our study, we use the k -shell decomposition method to partition the graph into a hierarchical structure. The main idea of k -shell decomposition is to divide the vertices into different layers by gradually removing the vertices with lower degrees in the graph.

We perform k -shell decomposition to obtain a k -shell layer set $\mathcal{G}^{ks} = \{V_1^{ks}, \dots, V_{m_{ks}}^{ks}\}$, where V_l^{ks} is the set of vertices in the l -th layer and $m_{ks} = |\mathcal{G}^{ks}|$ denotes the number of layers. Each layer $V_l^{ks} \in \mathcal{G}^{ks}$ has an attribute $d^{ks}(V_l^{ks})$, representing the vertex degree at the time of removal during the l -th layer decomposition. A layer with a higher d^{ks} value indicates vertices in this layer located in a denser region.

The process of k -shell decomposition is shown in [Algorithm 2](#). l , d_{min} , $\mathcal{G}^{ks}$ and V' are initialized accordingly, where l is the index of the current layer V_l^{ks} and d_{min} denotes the degree threshold for removing vertices from the current graph (Line 1). The algorithm selects all vertices with degrees less than or equal to d_{min} into V_l^{ks} and removes all those vertices from V' (Lines 4–5). If V_l^{ks} is not empty, it becomes a new layer to be added to $\mathcal{G}^{ks}$ (Line 7). The value of $d^{ks}(V_l^{ks})$ is updated to d_{min} , the value of w_l^{ks} is set to 0, and l is increased by 1 (Line 8). After that, the value of d_{min} is increased to repeatedly partition the graph until V' becomes empty (Line 9). It should be noted that the $d^{ks}(V_l^{ks})$ values of all layers are consecutive.

4.1.1. Weight update rule for $\mathcal{G}^{ks}$

Each layer $V_l^{ks} \in \mathcal{G}^{ks}$ is assigned a weight w_l^{ks} , which evaluates how frequently the layer contains vertices in the local best solutions. The weight w_l^{ks} for each layer in $\mathcal{G}^{ks}$ is initialized and updated according to the following rules. Initially, the w_l^{ks} value of every layer is set to 0. During the search, the update rule for w_l^{ks} is as follows.

Weight Update Rule 1. When the local best solution S_{lb_i} of the i -th worker thread improves, for each $V_l^{ks} \in \mathcal{G}^{ks}$, w_l^{ks} is increased by $|S_{lb_i} \cap V_l^{ks}|$.

The w_l^{ks} value is used to assist the secondary search mode in identifying promising regions and quickly constructing a high-quality solution, which is introduced in [Section 6.2](#).

Algorithm 1: ParaKplex algorithm.

Input: A graph $G = (V, E)$, the number of worker threads p , the parameter k and the *cutoff* time

Output: the global best solution S_{gb}

```

/* Master Thread */
1  $S_{gb} := \emptyset, \mathcal{G}^{ks} := Shell(G), \mathcal{G}^{sg} := SGGeneration(G, \mathcal{G}^{ks});$ 
/* Worker Threads in parallel */
2 for  $i = 1$  to  $p$  do in parallel
3    $S_{lb_i} := \emptyset, \alpha_i := 1, prefreq_i := iter_i := updfreq_i := 0;$ 
4   while elapsed time < cutoff do
5     for  $i = 1$  to  $p$  do in parallel
6       /* Independent Search Phase */
7       if  $i == 1$  then  $S_{lb_i} := SecoSearch(G, i, k);$ 
8       else  $S_{lb_i} := PrimSearch(G, i, S_{gb}, k);$ 
9        $iter_i := iter_i + 1;$ 
10      if  $iter_i \% T == 0$  then
11        if  $updfreq_i > T/2$  then  $\alpha_i := \alpha_i / 2;$ 
12        else  $\alpha_i := \alpha_i \times 2;$ 
13         $updfreq_i := 0;$ 
14      /* Interactive Sharing Phase */
15      if  $|S_{lb_i}| > |S_{gb}|$  then
16         $S_{gb} := S_{lb_i}$  and  $G := Reduce(G, S_{gb}, k);$ 
17        if  $G$  is empty or  $|V| \leq |S_{gb}|$  then return  $S_{gb};$ 
18      if  $\sum_{v \in V} freq_i(v) - prefreq_i \geq \alpha_i \times |V|$  then
19         $\mathcal{G}^{sg} := SGUpdate(G, \mathcal{G}^{sg}, i);$ 
20         $updfreq_i := updfreq_i + 1;$ 
21         $prefreq_i := \sum_{v \in V} freq_i(v) - prefreq_i;$ 
22 return  $S_{gb};$ 

```

4.2. Subgroup generation procedure

The purpose of the subgroup generation procedure is to partition the dense regions of the graph into subgroups and construct a subgroup set $\mathcal{G}^{sg}$. Previous work on the subgroup generation method begins by selecting a vertex with a high degree and iteratively adding candidates that connect to more vertices within the current subgroup [36]. However, this approach faces challenges when applied to large sparse graphs: the first vertex, despite having a high degree, may primarily connect to low-degree vertices, resulting in sparse regions. Consequently, constructing sufficiently large subgroups becomes difficult. To address this limitation, we incorporate layer information derived from the k -shell decomposition to generate subgroups within dense regions, thereby reducing exploration in sparse regions and improving subgroup quality.

The proposed subgroup generation procedure is detailed in Algorithm 3. We use V_j^{sg} to denote a subgroup in $\mathcal{G}^{sg}$, where j is the index of the current subgroup. Each subgroup $V_j^{sg} \in \mathcal{G}^{sg}$ is assigned a weight w_j^{sg} to measure which subgroup contains more promising vertices to construct a high-quality solution. Initially, the ungrouped vertex set (V') and the subgroup set ($\mathcal{G}^{sg}$) are initialized to an

Algorithm 2: Shell.**Input:** A graph $G = (V, E)$ **Output:** A k -shell layer set $\mathcal{G}^{ks}$

```

1  $l := 1, d_{min} := 0, \mathcal{G}^{ks} := \emptyset$  and  $V' := V$ ;
2 while  $V' \neq \emptyset$  do
3    $V_l^{ks} := \emptyset$ ;
4   while  $\exists v \in V'$  such that  $d_{G[V']}(v) \leq d_{min}$  do
5      $V_l^{ks} := V_l^{ks} \cup \{v\}$  and  $V' := V' \setminus \{v\}$ ;
6   if  $V_l^{ks} \neq \emptyset$  then
7      $\mathcal{G}^{ks} := \mathcal{G}^{ks} \cup \{V_l^{ks}\}$ ;
8      $d^{ks}(V_l^{ks}) := d_{min}, w_l^{ks} := 0$  and  $l := l + 1$ ;
9    $d_{min} := d_{min} + 1$ ;
10 return  $\mathcal{G}^{ks}$ ;
```

empty set, j is set to 0, and d_{avg}^{ks} denotes the average $d^{ks}(V_l^{ks})$ value over all layers $\mathcal{G}^{ks}$ (Line 1). The algorithm selects layers in the graph whose $d^{ks}(V_l^{ks})$ value exceeds the average d_{avg}^{ks} value and adds all vertices from these layers to V' (Lines 2–3). The algorithm uses a counter variable (*try*) to accumulate the number of consecutive times the subgroup fails to reach the size of Q_{init} , where parameter Q_{init} is the minimum required size for initially constructed subgroups. In our work, Q_{init} is set to 10. During subgroup generation (Lines 4–13), the layer with the largest $d^{ks}(V_l^{ks})$ value is identified, and a random vertex from this layer is chosen as the starting point for a new subgroup (Line 5). This vertex is then removed from V' and added to V_j^{sg} (Line 6). The candidate set (*Cand*) contains vertices in V' that ensure V_j^{sg} remains a k -plex when added to it (Line 7). While *Cand* is non-empty, vertices are iteratively added to V_j^{sg} to expand the subgroup (Lines 8–11). Once the size of V_j^{sg} reaches or exceeds parameter Q_{init} , the subgroup V_j^{sg} is stored in $\mathcal{G}^{sg}$, the w_j^{sg} value is set to 1, j is increased by 1, and *try* is reset to 0 (Lines 12–13). The algorithm terminates and returns $\mathcal{G}^{sg}$ when *try* reaches the predefined threshold *MaxTry* (Line 14).

4.2.1. Weight update rule for $\mathcal{G}^{sg}$

The weight w_j^{sg} for each subgroup in $\mathcal{G}^{sg}$ is initialized and updated according to the following rules. Initially, the w_j^{sg} of every subgroup is set to 1. During the search process, the update rule for w_j^{sg} is as follows.

Weight Update Rule 2. Suppose S_{ini} is the initial solution generated by the i -th worker thread based on $\mathcal{G}^{sg}$ or $\mathcal{G}^{ks}$. During the search process, if the current solution improves the local best solution S_{lbi} , the w_j^{sg} value of the subgroup including the first vertex added to S_{ini} is increased by 2, and the w_j^{sg} values of all the remaining subgroups in $\mathcal{G}^{sg}$ are increased by 1.

The rationale for this update rule is as follows. In the primary search, S_{ini} is an initial solution generated based on the group-based initialization procedure, and in the secondary search, it is a quickly constructed solution. When the current solution expanded from S_{ini} improves the local best solution S_{lbi} during the search, S_{ini} is regarded as a high-quality initial solution, as it effectively guides the search to reach a better solution. We consider that a subgroup V_j^{sg} containing the first vertex added to S_{ini} will include more vertices in S_{ini} . Specifically, since V_j^{sg} contains vertices from a dense subgraph, the first vertex selected from it is more likely to include its neighboring vertices within V_j^{sg} into the solution. Thus, the weight of V_j^{sg} should be increased by 2. To balance the weights of subgroups, the remaining subgroups are increased by 1. The w_j^{sg} value is used in our group-based initialization method, which is introduced in [Section 4.4](#).

4.3. Subgroup update procedure

To further optimize the subgroup set $\mathcal{G}^{sg}$, the algorithm dynamically refines it based on the local best solutions discovered by each worker thread. This subsection introduces a subgroup update procedure that considers two key factors: the connectivity of vertices within the subgroup and their potential to contribute to the best solution.

First, the connectivity of vertices within a subgroup is measured by their degree. Given a subgroup $V_j^{sg} \in \mathcal{G}^{sg}$, the adjacency $d_{G[V_j^{sg}]}(v)$ of a vertex $v \in V$ is defined as the number of vertices in V_j^{sg} that are connected to v , i.e., $d_{G[V_j^{sg}]}(v) = |N(v) \cap V_j^{sg}|$. Additionally, the average degree of the vertices in V_j^{sg} is denoted as $deg_{avg}(V_j^{sg})$, where $deg_{avg}(V_j^{sg}) = \frac{\sum_{v \in V_j^{sg}} d_{G[V_j^{sg}]}(v)}{|V_j^{sg}|}$. By comparing

Algorithm 3: SGGeneration.**Input:** A graph $G = (V, E)$ and k -shell layer set $\mathcal{G}^{ks}$ **Output:** A subgroup set $\mathcal{G}^{sg}$

```

1  $\mathcal{G}^{sg} := V' := \emptyset, j := 0$  and  $d_{avg}^{ks} = \frac{1}{m_{ks}} \sum_{l=1}^{m_{ks}} d^{ks}(V_l^{ks});$ 
2 foreach  $V_l^{ks} \in \mathcal{G}^{ks}$  do
3   if  $d^{ks}(V_l^{ks}) > d_{avg}^{ks}$  then  $V' := V' \cup V_l^{ks};$ 
4 for  $try := 0; try < MaxTry; try++$  do
5   select a layer  $V_l^{ks} \in \mathcal{G}^{ks}$  with the largest  $d^{ks}(V_l^{ks})$  value, and then randomly
   select a vertex  $v \in V_l^{ks};$ 
6    $V_j^{sg} := \{v\}$  and  $V' := V' \setminus \{v\};$ 
7    $Cand := \{u \in V' \mid |N(u) \cap V_j^{sg}| > |V_j^{sg}| - k\};$ 
8   while  $Cand \neq \emptyset$  do
9     randomly select a vertex  $v \in Cand;$ 
10     $V_j^{sg} := V_j^{sg} \cup \{v\}$  and  $V' := V' \setminus \{v\};$ 
11    update the elements in  $Cand$  accordingly;
12  if  $|V_j^{sg}| \geq Q_{init}$  then
13     $\mathcal{G}^{sg} := \mathcal{G}^{sg} \cup \{V_j^{sg}\}, w_j^{sg} := 1, j := j + 1, try := 0;$ 
14 return  $\mathcal{G}^{sg};$ 

```

$d_{G[V_j^{sg}]}(v)$ with $deg_{avg}(V_j^{sg})$, we can assess how well-connected vertex v is within the subgroup V_j^{sg} , reflecting its degree of connectivity to the subgroup.

Second, we assess the potential of a vertex $v \in V$ to be included in the best solution based on the ratio $\frac{step_i(v)}{freq_i(v)}$. Specifically, a higher value of $step_i(v)$ indicates that the vertex has remained in the current solution of the i -th worker thread for a longer time, suggesting a higher probability of being part of the best solution. Conversely, a higher value of $freq_i(v)$ indicates that a vertex is frequently removed from the solution, implying a lower likelihood of it being part of the best solution.

Considering both factors, we define the objective function for the subgroup update procedure as follows:

$$obj(\mathcal{G}^{sg}) = \sum_{V_j^{sg} \in \mathcal{G}^{sg}} \sum_{v \in V_j^{sg}} \left(d_{G[V_j^{sg}]}(v) + \frac{step_i(v)}{freq_i(v)} \right). \quad (1)$$

In the interactive sharing phase, the worker thread collects ungrouped vertices from the local best solution into V_{ungr} and aims to maximize the objective value $obj(\mathcal{G}^{sg})$. The rules for adding and removing vertices in the subgroup update procedure are as follows.

Subgroup Update Rule 1. Given a vertex $v \in V_{ungr}$, add this vertex into a subgroup V_j^{sg} that satisfies the following conditions: $d_{G[V_j^{sg}]}(v) \geq deg_{avg}(V_j^{sg})$ and $\frac{step_i(v)}{freq_i(v)} \geq \frac{\sum_{u \in V} step_i(u)}{\sum_{u \in V} freq_i(u)}$. If multiple subgroups satisfy these conditions, select the subgroup V_j^{sg} with the largest value of $d_{G[V_j^{sg}]}(v)$.

Subgroup Update Rule 2. Given a candidate subgroup $V_j^{sg} \in \mathcal{G}^{sg}$, remove several vertices in V_j^{sg} that satisfies the following conditions: $d_{G[V_j^{sg}]}(v) < deg_{avg}(V_j^{sg})$ and $\frac{step_i(v)}{freq_i(v)} < \frac{\sum_{u \in V} step_i(u)}{\sum_{u \in V} freq_i(u)}$.

The rationale for these update rules is as follows. In the subgroup update rule 1, to ensure the dense connectivity of subgroup vertices, the $d_{G[V_j^{sg}]}(v)$ of vertex v must reach or exceed the average degree of vertices in V_j^{sg} . Moreover, to ensure that v has a high probability of being included in the best solution, the ratio of $step_i(v)$ to $freq_i(v)$ must be greater than or equal to the total ratio of the sum of $step_i$ to $freq_i$ of all vertices in V . The condition for the subgroup update rule 2 is opposite, vertices with a low d_G value and a low ratio of $step_i$ to $freq_i$ are removed from the candidate subgroup.

The subgroup update procedure is presented in Algorithm 4. We use the lower bound Q_{low} and the upper bound Q_{up} to denote the minimum and maximum size of a subgroup, respectively. Both Q_{low} and Q_{up} are calculated based on the average size of the current subgroup set, where β_1 and β_2 are parameters (Line 1). During the removal process, the algorithm traverses all subgroups in $\mathcal{G}^{sg}$ and removes vertices based on the subgroup update rule 2 (Line 3). If the size of V_j^{sg} falls below the lowerbound Q_{low} , the

Algorithm 4: SGUpdate.

Input: A graph $G = (V, E)$, the subgroup set $\mathcal{G}^{sg}$, and the i -th worker thread

Output: A updated subgroup $\mathcal{G}^{sg}$

```

1  $Q_{low} = \frac{\sum_{V_j^{sg} \in \mathcal{G}^{sg}} |V_j^{sg}|}{|\mathcal{G}^{sg}|} \times \beta_1$  and  $Q_{up} = \frac{\sum_{V_j^{sg} \in \mathcal{G}^{sg}} |V_j^{sg}|}{|\mathcal{G}^{sg}|} \times \beta_2$ ;
2 foreach subgroup  $V_j^{sg} \in \mathcal{G}^{sg}$  do
3   removes vertices from  $V_j^{sg}$  based on Subgroup Update Rule 2;
4   if  $|V_j^{sg}| < Q_{low}$  then  $\mathcal{G}^{sg} := \mathcal{G}^{sg} \setminus \{V_j^{sg}\}$ ;
5  $V_{ungr} := S_{lb_i} \setminus \bigcup_{V_j^{sg} \in \mathcal{G}^{sg}} V_j^{sg}$ ;
6 foreach vertex  $v \in V_{ungr}$  do
7   add  $v$  into a subgroup  $V_j^{sg} \in \mathcal{G}^{sg}$  based on Subgroup Update Rule 1;
8 foreach subgroup  $V_j^{sg} \in \mathcal{G}^{sg}$  do
9   if  $|V_j^{sg}| > Q_{up}$  then
10    select a vertex  $v \in V_j^{sg}$  with the smallest  $d_{G[V_j^{sg}]}(v)$  value, breaking ties
11    randomly;
12     $V_j^{sg} := V_j^{sg} \setminus \{v\}$  and  $V_{new}^{sg} := \{v\}$ ;
13     $Cand := N(v) \cap V_j^{sg}$ ;
14    while  $|V_{new}^{sg}| < Q_{low}$  &&  $Cand \neq \emptyset$  do
15      select a vertex  $u \in Cand$  with the smallest  $d_{G[V_j^{sg}]}(u)$  value, breaking
16      ties randomly;
17       $V_j^{sg} := V_j^{sg} \setminus \{u\}$  and  $V_{new}^{sg} := V_{new}^{sg} \cup \{u\}$ ;
18       $Cand := Cand \cap N(u)$ ;
19    if  $|V_{new}^{sg}| \geq Q_{low}$  then
20       $\mathcal{G}^{sg} := \mathcal{G}^{sg} \cup \{V_{new}^{sg}\}$  and  $w_{new}^{sg} := 1$ ;
21    else  $V_j^{sg} := V_j^{sg} \cup V_{new}^{sg}$ ;
22 return  $\mathcal{G}^{sg}$ ;
```

algorithm removes V_j^{sg} from $\mathcal{G}^{sg}$ (Line 4), as such subgroups provide insufficient guidance for initialization. We use V_{ungr} to denote the vertices in S_{lb_i} that are not contained in any subgroup in $\mathcal{G}^{sg}$ (Line 5). The algorithm adds the ungrouped vertices from V_{ungr} to the corresponding subgroups according to the subgroup update rule 1 (Lines 6–7). For subgroups whose size exceeds Q_{up} , the algorithm should redivide them (Lines 8–19). The algorithm selects a vertex v with the smallest $d_{G[V_j^{sg}]}(v)$ value from the current subgroup V_j^{sg} to construct a new subgroup V_{new}^{sg} , and removes v from V_j^{sg} (Lines 10–11). We use the $Cand$ to store all the candidate vertices connected to the vertex v in V_j^{sg} (Line 12). The algorithm iteratively constructs V_{new}^{sg} until it becomes a new subgroup (Lines 13–16). If the size of V_{new}^{sg} reaches Q_{low} , then V_{new}^{sg} is added to $\mathcal{G}^{sg}$ and the weight of the new subgroup V_{new}^{sg} is set to 1; otherwise, the vertices in V_{new}^{sg} are added back to V_j^{sg} (Lines 17–19). Finally, the algorithm returns the updated subgroup set $\mathcal{G}^{sg}$ (Line 20).

4.4. Group-based initialization procedure

In this subsection, we propose a group-based initialization procedure and describe how to leverage the subgroup set $\mathcal{G}^{sg}$ and historical search information to generate high-quality initial solutions for each worker thread.

Before introducing the group-based initialization method, we first explain the roulette wheel selection method. The roulette wheel selection method is a stochastic approach where the selection probability of an individual is proportional to its fitness [43]. In our

algorithm, the w_j^{sg} value of a subgroup serves as its fitness. Specifically, for all subgroups in $\mathcal{G}^{sg}$, the selection probability of a subgroup V_j^{sg} is calculated as $\frac{w_j^{sg}}{\sum_{V_j^{sg} \in \mathcal{G}^{sg}} w_j^{sg}}$.

Algorithm 5: GBInitialization.

Input: A graph $G = (V, E)$ and the i -th worker thread

Output: An initial solution S

```

1  $S = \emptyset$ ;
2 if  $\mathcal{G}^{sg} \neq \emptyset$  with probability  $pro$  then
3   select a subgroup  $V_j^{sg} \in \mathcal{G}^{sg}$  based on the roulette wheel selection, and then
   randomly select a vertex  $v \in V_j^{sg}$ ;
4    $S := S \cup \{v\}$ ;
5    $Cand := \{u \in V_j^{sg} \mid |N(u) \cap S| > |S| - k\}$ ;
6   while  $Cand \neq \emptyset$  do
7     if  $Cand \cap \left(\bigcup_{V_j^{sg} \in \mathcal{G}^{sg}} V_j^{sg}\right) \neq \emptyset$  then
8       select a vertex  $v \in Cand \cap \left(\bigcup_{V_j^{sg} \in \mathcal{G}^{sg}} V_j^{sg}\right)$  with the smallest  $freq_i$ 
       value;
9     else select a vertex  $v \in Cand$  with the smallest  $freq_i$  value ;
10     $S := S \cup \{v\}$ ;
11    update the elements in  $Cand$  accordingly;
12 else  $S := Init_{freq}(G)$  ;
13 return  $S$ ;
```

The group-based initialization procedure, outlined in Algorithm 5, begins by initializing the solution S as an empty set (Line 1). If the input instance fails to form a subgroup set or with probability $1 - pro$, the algorithm constructs the initial solution using $Init_{freq}$ (Line 12). In detail, the $Init_{freq}$ procedure is the frequency-based initialization method [41]. It iteratively selects a vertex with the smallest $freq_i$ value in the candidate set and then adds it into S until the candidate set becomes empty. Otherwise, the algorithm employs the roulette wheel selection method to choose a subgroup from $\mathcal{G}^{sg}$ and selects a random vertex from the chosen subgroup (Line 3), which is then added to S (Line 4). The set $Cand$ is defined to contain vertices from G that satisfy the k -plex constraint and can be added to S (Line 5). While $Cand$ is not empty, the algorithm preferentially selects a vertex with the smallest $freq_i$ value among those contained in any subgroup in $\mathcal{G}^{sg}$ (Line 8). If no such vertex exists, it selects the vertex with the smallest $freq_i$ value from $Cand$ (Line 9). The selected vertex is added to S , and $Cand$ is updated accordingly (Lines 10–11). The process terminates when $Cand$ becomes empty, returning the constructed initial solution S (Line 13).

Compared with previous work [36], our initialization method has two differences. First, the previous work selects the first vertex from a random subgroup, our algorithm selects the first vertex from a subgroup with high w_j^{sg} . A high value of w_j^{sg} indicates that the initial solution generated within this subgroup has a greater likelihood of improving the local best solution recorded in each worker thread. Second, during the construction of the initial solution, the previous work iteratively selects a vertex from the candidate set with higher connectivity to the expanding solution. Our algorithm explores rarely visited vertices by adding vertices with low $freq_i$ values.

5. Three novel sequential strategies

In this section, we propose three sequential strategies including a configuration checking strategy, a vertex selection strategy, and a perturbation strategy to improve the search efficiency. To enhance clarity for the reader, we omit the index of worker thread i in the representation of the variables in this section, as all of them are private to each worker thread.

5.1. Relaxed bounded configuration checking strategy

Configuration Checking (CC) is a widely adopted diversification strategy in local search designed to prevent cycling [39]. In the CC strategy, a vertex $v \notin S$ is forbidden from being added to S , if its configuration has remained unchanged since v 's last removal from S . In general, a vertex's configuration refers to the state of its adjacent vertices. The CC strategy employs an array $conf$ to represent a vertex's configuration: $conf(v) = 1$ indicates that v is allowed to be added to the candidate solution, and $conf(v) = 0$ indicates that it is forbidden.

5.1.1. Review of CC strategies for clique relaxation problems

Recently, three variants of CC have been proposed for solving the relaxed clique problem, including dynamic-threshold configuration checking (DCC) [5], bounded configuration checking (BoundedCC) [7] and stratified threshold configuration checking (STCC) [44]. The CC strategies mentioned above all maintain an integer threshold $thred$ to control the forbidding strength and $conf$ to count the times of configuration changes. Only when $conf(v) \geq thred(v)$, v is allowed to be added back to S . We review the update rules for these strategies as follows.

In DCC, $conf(v)$ and $thred(v)$ are initialized to 1 for each vertex. When a vertex v is added to S , $conf(v)$ is reset to 0 and $thred(v)$ is increased by 1. The $conf$ values of v 's neighbors are also increased by 1. When v is removed from S , its $conf(v)$ is reset to 0.

In BoundedCC, $conf(v)$ and $thred(v)$ are initialized to 1 for each vertex. When a vertex v is added to S , $thred(v)$ is increased by 1, and the $conf$ values of v 's neighbors are also increased by 1. When the value of $thred$ reaches the predefined upper bound ub_thre , it will be reset to 1. When v is removed from S , its $conf(v)$ is reset to 0.

In STCC, $conf(v)$ and $thred(v)$ are initialized to k for each vertex. When a vertex v is added to S , $thred(v)$ is increased by 1, and the $conf$ values of v 's neighbors are increased by a value determined by the specific features of the maximum k -club problem. When v is removed from S , its $conf(v)$ is reset to 0.

5.1.2. Our proposed RBCC strategy

For a vertex v , when $thred(v)$ reaches the upper bound ub_thre , indicating that v has been forbidden for a period of time, BoundedCC just resets its $thred$ value to 1. Preliminary experiments indicate that this updating strategy can mislead the search by prohibiting promising candidate vertices. To overcome this issue, we design a novel update rule that leverages vertex frequency as search information.

Suppose a vertex v is added into the candidate solution and $thred(v)$ exceeds ub_thre . In such a scenario, if $freq(v)$ is not too high compared to other vertices in the candidate set, its restrictive condition should be relaxed further. Specifically, we randomly sample t vertices to form a candidate set $FreqSet = \{v_1^f, v_2^f, \dots, v_t^f\}$. The vertices in $FreqSet$ are arranged in a descending order of their $freq$ values. If $freq(v)$ is less than the 80th percentile of the values in $FreqSet$, i.e., the value at position $\lfloor 0.8 \times t \rfloor$ in the sorted list of $freq$ values, then vertex v should be relaxed, and $thred(v)$ should be set to 0.

Building on the rule described above, we propose a more relaxed strategy named the relaxed bounded configuration checking (RBCC) strategy. The update rules of our strategy are as follows.

RBCC Initial Rule. At the beginning, for each $v \in V$, $conf(v)$ and $thred(v)$ are initialized to 1.

RBCC Add Rule. When v is added into the candidate solution, $thred(v)$ and $conf(v')$ are increased by 1 for each $v' \in N(v)$. If $thred(v) \geq ub_thre$, we randomly select t vertices to generate a frequency candidate set $FreqSet = \{v_1^f, v_2^f, \dots, v_t^f\}$. If $freq(v) < freq(v_{\lfloor 0.8 \times t \rfloor}^f)$, then $thred(v)$ is reset to 0; Otherwise, $thred(v)$ is reset to 1.

RBCC Swap Rule. When u is removed and v is added into the candidate solution, $conf(u)$ is switched to 0.

RBCC Drop Rule. When removing a vertex u from the candidate solution, $conf(u)$ is set to 0.

5.1.3. Two constrained candidate sets

During the search, the Add operation can directly enhance the quality of the candidate solution, while the Swap operation serves as a form of diversification by guiding the search toward a new neighboring candidate solution. Consequently, we argue that vertices in the $AddSet$ should be encouraged to be added to the candidate solution. Unlike DCC, BoundedCC, and STCC, which impose uniform CC constraints on both Add and Swap operations, we employ a classic tabu mechanism [40] as an auxiliary method to relax the CC constraint specifically for the Add operation. For each vertex $v \in V$, $age(v)$ denotes the number of steps since v was last removed from the candidate solution. The candidate sets for the Add and Swap operations are as follows:

- $ConstrAddSet(S) = \{v \in AddSet(S) \mid conf(v) \geq ub_thre\} \cup \{v \in AddSet(S) \mid age(v) > L_t\};$
- $ConstrSwapSet(S) = \{v \in SwapSet(S) \mid conf(v) \geq ub_thre\}.$

Here, L_t is a tabu parameter specifying the number of steps during which vertices outside $AddSet$ cannot be added back.

5.2. A solution information-based vertex selection strategy

To select high-quality candidate vertices, we propose a novel vertex selection strategy that leverages both the current solution information and previously generated local best solutions.

5.2.1. Previous vertex selection strategies

In the following, we review two vertex selection strategies, namely random-based and structure-based methods.

The random-based method is employed in earlier algorithms such as FD-TS [37] and DCCplex [5]. These algorithms typically utilize tabu [40] or CC [39] to tackle the cycling problem. In the adding and swapping processes, if there are any vertices in the corresponding candidate set, the algorithm randomly selects one from it.

The structure-based method has been employed in KLS [34]. In this algorithm, a tabu mechanism is utilized to reduce the candidate set, and a scoring function, denoted as $d_{G[S]}$, is used to select the promising vertex. Given a candidate solution $S \subseteq V$, the adjacency $d_{G[S]}(v)$ of a vertex $v \in N(S)$ represents the number of vertices in S connected to v , and is formally defined as $d_{G[S]}(v) = |N(v) \cap S|$. The algorithm selects a candidate vertex that satisfies the tabu constraint and with the largest $d_{G[S]}$ value, further ties are broken by preferring the one with the minimum $freq$ value.

5.2.2. Current solution information-based scoring function

In this subsection, we design a novel scoring function based on the graph structure and the current solution.

It is evident that adding a vertex with large $d_{G[S]}$ value typically results in fewer saturated vertices. Therefore, we primarily use $d_{G[S]}$ as the basis for designing our first scoring function. Additionally, $\overline{d_{G[S]}}(v) = d_G(v) - d_{G[S]}(v)$ represents the number of vertices in $V \setminus S$ adjacent to v once it is added to S . The value of $\overline{d_{G[S]}}(v)$ indicates the potential for expanding S after adding v . We propose our scoring function as follows:

$$score_{csi}(v) = d_{G[S]} \times \left(1 + \frac{\rho \times \overline{d_{G[S]}}(v)}{|V|}\right). \quad (2)$$

Here, ρ is a parameter ranging from 0 to 1. On the one hand, according to the definition of $score_{csi}$, $d_{G[S]}(v)$ has the greatest influence on $score_{csi}$. On the other hand, since we use $\frac{\rho \times \overline{d_{G[S]}}(v)}{|V|}$ as the coefficient for $d_{G[S]}(v)$, $\overline{d_{G[S]}}$ can be regarded as the second factor influencing $score_{csi}$ and $\frac{\rho}{|V|}$ serves to balance the influence between $d_{G[S]}$ and $\overline{d_{G[S]}}$.

The intuition behind $score_{csi}$ is to leverage the information from the current solution. Specifically, it accounts for both the immediate impact (i.e., $d_{G[S]}$) and the subsequent extendibility (i.e., $\overline{d_{G[S]}}$) when a vertex is added to S .

5.2.3. Local best solution information-based scoring function

The proposed $score_{lbs}$ function is designed based on information from historical local best solutions, and its update rules are as follows.

Initial Rule. At the beginning, for each $v \in V$, $score_{lbs}(v)$ is initialized to 1.

Update Rule. When the local search procedure returns a local best solution S_{lb} , for each $v \in V$, $score_{lbs}(v)$ is increased by $d_{G[S_{lb}]}(v)$.

The design of $score_{lbs}$ is inspired by the backbone structure in combinatorial optimization problems [45], which suggests that high-quality solutions often share certain common components. The intuitive criterion for evaluating the backbone is the number of times a variable appears in the local best solutions. In our work, we extend this idea to the cumulative number of connections between v and each local best solution S_{lb} .

5.2.4. Our proposed vertex selection strategy

Based on the above two scoring functions, we design two novel vertex selection rules as follows.

Add Rule. Add one vertex $v \in ConstrAddSet(S)$ with the highest $score_{csi}(v)$ to S , with ties broken randomly.

Swap Rule. When $dens(G) > \alpha$, a vertex $v \in ConstrSwapSet(S)$ with the highest $score_{lbs}(v)$ is selected into S , with ties broken randomly. Otherwise, when $dens(G) \leq \alpha$, a vertex $v \in ConstrSwapSet(S)$ with the lowest $freq(v)$ is selected, with ties broken randomly.

According to the preliminary experiment, the parameter α is set to 0.35. In the Swap operation, after adding vertex v , we remove a vertex from S that is not connected to v . The method for selecting a vertex to remove from $SwapSet(S)$ was described in Section 2.

For the Add operation, since adding a vertex can directly enhance the solution quality, we adopt a greedy approach based on the $score_{csi}$ value to select the candidate vertex to be added, ensuring the expandability of the candidate solution after the addition. Regarding the Swap operation, which aims to move the current candidate solution to its neighboring search space, we leverage search information to explore the search spaces based on the structural properties of the given graph (e.g., density). Specifically, if the graph is highly sparse, the search procedure tends to get trapped in local optima. Thus, we focus on exploring rarely visited vertices using the $freq$ value. Conversely, for dense graphs, we focus on exploring potential high-quality vertices by considering the $score_{lbs}$ value.

5.3. A core-based perturbation strategy

Although our RBCC strategy can deal with the cycling problem, it struggles to escape local optima. Therefore, we introduce a core-based perturbation strategy based on the definition of the solution core. This strategy consists of shrinking and expanding phases. In the shrinking phase, the candidate solution is refined to a solution core. The expanding phase generates a new perturbed solution by extending the solution core. We first give the definition of the solution core as follows.

Definition 1. (Solution Core) Given a graph $G = (V, E)$, an integer $k \geq 2$ and a k -plex solution S , a solution core of S is a subset $S_{core} \subseteq S$ such that for each $v \in S_{core}$, $|N(v) \cap S_{core}| > |S_{core}| - k$.

When our algorithm searches for a k -plex, given a candidate solution S , the solution core S_{core} is defined as a subset of S that satisfies the constraints of a $(k-1)$ -plex. S_{core} is a more cohesive structure than S .

Based on the definition of solution core, the core-based perturbation strategy is presented in Algorithm 6. In the shrinking phase (Lines 2–5), a solution core can be obtained by iteratively removing a vertex from the saturated vertices set $C(S)$ until $C(S)$ becomes empty. During this phase, all removed vertices are stored in a set $TempSet$. In the expanding phase (Lines 6–12), to obtain a perturbed solution S , the algorithm iteratively selects a vertex in $AddSet \cap TempSet$ with probability p and otherwise it selects a vertex in $AddSet \setminus TempSet$ until one of the above two sets becomes empty. Finally, the perturbed solution is returned (Line 13).

Algorithm 6: CorePerturb.

Input: the candidate solution S

Output: the perturbation solution S

```

1  $TempSet := \emptyset$ ;
2 while  $C(S) \neq \emptyset$  do
3   randomly select a saturated vertex  $u \in C(S)$ ;
4    $S := S \setminus \{u\}$  and  $TempSet := TempSet \cup \{u\}$ ;
5   update the  $conf(u)$  value based on RBCC Drop Rule;
6 while  $AddSet \cap TempSet \neq \emptyset$  &&  $AddSet \setminus TempSet \neq \emptyset$  do
7   if with probability  $p$  then
8     randomly select a vertex  $v \in AddSet \cap TempSet$ ;
9   else
10    randomly select a vertex  $v \in AddSet \setminus TempSet$ ;
11    $S := S \cup \{v\}$ ;
12   update the  $conf$  and  $thred$  values of  $v$  and its neighbors according to RBCC Add Rule;
13 return  $S$ ;
```

6. A primary-secondary search method

In this section, we propose a primary-secondary search method, and describe the details and differences between the primary and secondary search.

6.1. The primary search mode

The primary search mode iteratively explores the search space through Add, Swap, and Drop operations guided by the RBCC strategy, and triggers a perturbation strategy to escape local optima when the number of unimproved steps reaches a threshold. The details are presented in Algorithm 7.

For each worker thread, the algorithm initializes the $conf_i$ and $thred_i$ accordingly (Line 1). Then, the algorithm calls the group-based initialization method to generate an initialization solution S_{ini} and initializes S_i and S_{lb_i} to S_{ini} , with the unimproved step counter $unimpr_i$ set to 0 (Line 2). Subsequently, the local search procedure is applied to improve the current solution S_i (Lines 3–17). Finally, the algorithm returns the local best solution S_{lb_i} that was found (Line 18).

In the local search procedure, cur_step_i records the current number of iteration steps, and L denotes the search depth (Line 3). In each iteration, the algorithm sequentially checks $ConstrAddSet$ and $ConstrSwapSet$. If $ConstrAddSet$ is not empty, a vertex v is added to S_i according to the add rule (Lines 4–5). Otherwise, if $ConstrSwapSet$ is not empty, the algorithm exchanges a pair of vertices based on the swap rule (Lines 6–7). If these two sets are empty, a random vertex is removed from S_i (Lines 8–9). The values of $thre_i$ and $conf_i$ are updated accordingly based on the corresponding RBCC rules (Line 10). The $score_{lbs}$ value of each vertex is updated according to the information of S_{lb_i} (Line 11). If S_i outperforms S_{lb_i} , the algorithm updates S_{lb_i} by S_i and $unimpr_i$ is reset to 0 (Line

13). A worker thread holds write locks to update the weights of w_j^{sg} in $\mathcal{G}^{sg}$ and w_l^{ks} in $\mathcal{G}^{ks}$ according to the weight update rules 1 and 2, respectively (Line 14). After that, the algorithm checks whether the unimproved step $unimpr_i$ reaches $k \times |S_{gb}|$ (Line 15). If so, the algorithm performs the core-based perturbation strategy and resets $unimpr_i$ to 0 (Line 16). At the end of each round, $unimpr_i$ is increased by 1 (Line 17).

Algorithm 7: PrimSearch.

Input: A graph $G = (V, E)$, the i -th worker thread, the global best solution S_{gb} , and the parameter k

Output: A local best solution S_{lb_i}

```

1 initialize the  $conf_i$  and  $thred_i$  values of each vertex based on RBCC Initial Rule;
2  $S_{lb_i} := S_i := S_{in_i} := G\text{BInitialization}(G, i)$ ,  $unimpr_i := 0$ ;
3 for  $cur\_step_i := 0$ ;  $cur\_step_i < L$ ;  $cur\_step_i++$  do
4   if  $ConstrAddSet(S_i) \neq \emptyset$  then
5     select an added vertex  $v$  based on Add Rule and  $S_i := S_i \cup \{v\}$ ;
6   else if  $ConstrSwapSet(S_i) \neq \emptyset$  then
7     select an added vertex  $v$  and a removed vertex  $u$  based on Swap Rule and
        $S_i := S_i \setminus \{u\} \cup \{v\}$ ;
8   else
9     randomly select a removed vertex  $u$  from  $DropSet$  and  $S_i := S_i \setminus \{u\}$ ;
10  update the  $conf_i$  values of the vertices based on the corresponding RBCC Rule;
11  update the related scoring function values based on Update Rule;
12  if  $|S_i| > |S_{lb_i}|$  then
13     $S_{lb_i} := S_i$  and  $unimpr_i := 0$ ;
14    update  $w_l^{ks}$  and  $w_j^{sg}$  based on  $S_{lb_i}$  and  $S_{in_i}$  accordingly;
15  if  $unimpr_i > k \times |S_{gb}| \ \&\& \ S_{gb} \neq \emptyset$  then
16     $S_i := CorePerturb(S_i)$  and  $unimpr_i := 0$ ;
17   $unimpr_i := unimpr_i + 1$ ;
18 return  $S_{lb_i}$ ;

```

6.2. The secondary search mode

The secondary search mode aims to select a vertex from a high weight layer and then greedily expand it to construct a solution. The details of this mode are presented in Algorithm 8.

The algorithm samples half of layers from $\mathcal{G}^{ks}$, and selects a layer V_l^{ks} with the largest w_l^{ks} value among them (Line 1). Then, a vertex is randomly chosen from the layer V_l^{ks} and added to S_{in_i} (Line 2). We use $Cand_i$ to store the candidate vertices that satisfy the constraints of k -plex (Line 3). Subsequently, the solution S_{in_i} is extended iteratively by adding a vertex $v \in Cand_i$, until $Cand_i$ becomes empty (Lines 4–7). Each time, the algorithm selects a vertex that can form more potential candidate vertices to join S_{in_i} (Line 5). $|S_{in_i} \cup \{v\}| + |N(v) \cap Cand_i|$ represents the upper bound that can expand vertices by adding vertex v to S_{in_i} (Line 6). S_{lb_i} is updated to S_{in_i} if the size of S_{in_i} exceeds that of S_{lb_i} (Line 9). A worker thread updates the weights of w_l^{ks} and w_j^{sg} according to the corresponding rules by holding write locks (Line 10). Finally, the algorithm returns S_{lb_i} (Line 11).

6.3. Differences between primary and secondary search modes

The primary search mode employs three operations to update the candidate solution, with the goal of finding the global best solution. In contrast, the secondary search mode omits the Swap and Drop operations. The purpose of the secondary search mode is to quickly construct a high-quality solution, offering two key advantages that facilitate the interactive sharing phase.

Algorithm 8: SecoSearch.**Input:** A graph $G = (V, E)$, the i -worker thread, and the parameter k **Output:** a local best solution S_{lb_i}

- 1 select the layer V_l^{ks} with the highest w_l^{ks} value from half of the randomly selected layers in $\mathcal{G}^{ks}$;
- 2 select a random vertex $v \in V_l^{ks}$ and $S_{in_i} := \{v\}$;
- 3 $Cand_i := \{v \in V \mid |N(v) \cap S_{in_i}| > |S_{in_i}| - k\}$;
- 4 **while** $Cand_i \neq \emptyset$ **do**
 - 5 select a vertex $v \in Cand_i$ with the largest $|N(v) \cap Cand_i|$ value, breaking ties randomly;
 - 6 **if** $|S_{in_i} \cup \{v\}| + |N(v) \cap Cand_i| \leq |S_{lb_i}|$ **then break**;
 - 7 $S_{in_i} := S_{in_i} \cup \{v\}$, $Cand_i := Cand_i \cap N(v)$;
- 8 **if** $|S_{in_i}| > |S_{lb_i}|$ **then**
 - 9 $S_{lb_i} := S_{in_i}$;
 - 10 update w_l^{ks} and w_j^{sg} based on S_{lb_i} and S_{in_i} accordingly;
- 11 **return** S_{lb_i} ;

First, due to its greedy nature, the secondary search mode can quickly identify a solution. This solution can also be used to reduce the given graph to a certain extent, allowing both search modes to operate within a smaller search space.

Second, the secondary search mode can identify a variety of high-quality solutions (i.e., different regions within the search space). These solutions are then shared to update the subgroup set $\mathcal{G}^{sg}$, ensuring that promising vertices from dense regions are included in $\mathcal{G}^{sg}$. This process creates more high-quality starting points for the primary search mode.

7. Differences between ParaKplex and nukplex

ParaKplex is an improved version of the Nukplex algorithm, which was originally published in our conference paper [41]. In this section, we discuss two main differences between ParaKplex and Nukplex as follows.

- ParaKplex is an algorithm that incorporates two complementary search modes, whereas Nukplex utilizes only a single mode. ParaKplex adopts the same search mode as Nukplex as its primary search mode to thoroughly explore the entire search space. Additionally, it employs a secondary search mode to efficiently generate solutions in dense regions. Once either search completes, the local best solution is used to update the global best solution, reduce the given graph, and update the subgroups. The updated subgroup information and search information are used to facilitate the collaborative search of these two modes.
- ParaKplex employs a novel group-based initialization method, while Nukplex utilizes a frequency-based method. When constructing the initial solution, ParaKplex not only intensively exploits promising regions based on the subgroup set to construct high-quality initial solutions, but also selects candidate vertices with low selection frequency to guide the search process toward unvisited search spaces.

8. Experimental evaluation

In this section, we first introduce the selected benchmarks and the adopted experimental setup. Then, we conduct extensive experiments to test and report the performance of our algorithm in comparison with other algorithms on a broad range of benchmarks. We also perform further empirical analyses to highlight the effectiveness of our proposed strategies and the robustness of our algorithm. Finally, we evaluate the statistical differences between our algorithm and the comparison algorithms for each benchmark.

8.1. Benchmarks and competitors

To comprehensively evaluate the performance of our algorithm and competitors, we collected all the benchmark datasets previously used to evaluate the MKPP algorithms [5,12,13,25,26,34]. These benchmarks include two classic benchmarks and three benchmarks for large sparse graphs. Next, we will first introduce the two classic benchmarks.

- **BHOSLIB**¹: This dataset consists of 41 graphs, which were generated randomly in the phase transition area according to the model RB [46]. The sizes of these graphs with the number of vertices ranging from 4×10^2 to 1.5×10^3 , and the number of edges ranges from 1.7×10^4 to 1.2×10^6 .
- **DIMACS**²: This dataset of 80 graphs originated from the second DIMACS implementation challenge for the maximum clique problem [47]. The graphs in this dataset can have up to 4×10^3 vertices and up to 5.5×10^6 edges, with densities ranging from 0.03 to 0.99.

The following describes the three large sparse graph benchmarks used in our experiments.

- **Real-world**³: This dataset contains 139 real-world graphs, selected from the Network Data Repository [48], and includes various types of networks such as social, biological, collaboration, and so on. The graphs within this dataset have up to 5.87×10^7 vertices and up to 10^8 edges.
- **DIMACS10**⁴: This dataset contains 84 graphs from the 10th DIMACS Implementation Challenge. This dataset has been widely used to evaluate graph algorithms such as the maximum clique and the maximum k -plex problems. The graphs in this dataset have up to 2×10^7 vertices and up to 10^8 edges.
- **SNAP**⁵: This dataset includes 53 graphs from the Stanford Large Network Dataset Collection [49], encompassing a variety of networks such as communication networks, social networks, and Wikipedia networks. The graph in this dataset has more than 10^7 vertices and more than 2×10^8 edges.

To evaluate the effectiveness of ParaKplex, we compare it with three heuristic algorithms (GPULS [35], KLS [34] and DCCplex [5]) and six exact algorithms (DiseMKP [25], KpLeX-Gap [26], kPEX [13], KplexT [12], RelaxPUB-Maplex [27] and U-MKP [28]).

- **GPULS**⁶: It is a GPU-based local search algorithm. GPULS is applied to solve both the weighted and unweighted MKPP, and two novel neighborhood structures are designed to enable efficient parallel search.
- **KLS**: It is an effective k -plex local search algorithm. KLS adopts a data structure suitable for graph analysis and has mechanisms for preventing the repeated revisiting of the same solutions during the search and promoting search diversity.
- **DCCplex**: It is a local search algorithm. DCCplex employs a dynamic-threshold configuration checking strategy to avoid repeatedly revisiting the same solutions during the search, along with an incremental updating scheme based on the number of non-neighbors in the current candidate solution and saturated vertex set to efficiently identify feasible neighborhoods.
- **DiseMKP**⁷: It is a branch and bound algorithm. DiseMKP enhances the partition-based upper bound method [24] to achieve tighter bounds for pruning the search space, and employs an inprocessing strategy to incrementally reduce graphs.
- **KpLeX-Gap**⁸: It is an effective branch and bound algorithm. KpLeX-Gap calculates the gap between the graph's degeneracy bound and the maximum k -plex size to determine if certain subgraphs may contain a k -plex, and uses graph reduction and bound estimation techniques to reduce the search space.
- **kPEX**: It is a branch and bound algorithm. kPEX first partitions a branch into two parts, then alternatively conducts the reduction process and the bounding process in each branch. It also develops pre-processing methods to reduce the size of the graph.
- **KplexT**⁹: It is a recent branch and bound algorithm. KplexT employs a two-stage framework. In the first stage, it incorporates reduction and branching rules to reduce the time complexity of the search. In the second stage, it combines an initialization method and bounding technique to optimize the algorithm.
- **RelaxPUB-Maplex**¹⁰: It is a recent branch and bound algorithm. RelaxPUB-Maplex combines the complementary strengths of graph coloring and graph partitioning techniques to design a novel upper bound method.
- **U-MKP**¹¹: It is a parallel branch and bound algorithm proposed recently. The algorithm framework integrates the various pruning techniques including branching, upper bounding, and reduction strategies from different BnB algorithms.

8.2. Experiment preliminaries

Our computational assessment was conducted on classic and large sparse graph benchmarks, including 397 graphs. The value of k is usually small in previous studies, and only a few works have the maximum value of k to 20 [13,41]. In our study, we tested the used k values (i.e., $k = 2, 3, 4, 5, 10, 15, 20$) in previous work [12–14] and increased the k values to 25 and 30 to discover larger cohesive subgroups [11]. Therefore, we consider nine different values of k and test 3573 instances in total.

¹ <https://networkrepository.com/bhoslib.php>

² <http://archive.dimacs.rutgers.edu/pub/challenge/graph/>

³ <http://networkrepository.com/networks.php>

⁴ <https://networkrepository.com/dimacs10.php>

⁵ <http://snap.stanford.edu/data>

⁶ <https://sites.google.com/site/nogueirabruno/software>

⁷ <https://github.com/huajiang-ynu/ijcai23-kpx>

⁸ https://github.com/joey001/kplex_degen_gap

⁹ <https://lijunchang.github.io/Maximum-kPlex-v2/>

¹⁰ <https://github.com/JHL-HUST/RelaxPUB>

¹¹ <https://github.com/akhlaqueak/MKP-Study/tree/main>

Table 1
Tuned parameters of the ParaKplex algorithm.

Parameter	Range	Final value
Group-Based Search		
β_1	{0.6, 0.7, 0.8, 0.9}	0.8
β_2	{1.0, 1.5, 2.0, 2.5}	2.0
pro	{0.5, 0.6, 0.7, 0.8}	0.6
$MaxTry$	{5, 10, 15, 20}	10
T	{10, 20, 30, 40}	20
Some other parameters in local search		
t	{40, 50, 60, 70}	50
L_i	{3, 4, 5}	4
ub_thre	{2, 3, 4, 5}	3
ρ	{0, 0.1, 0.2, 0.3}	0.2
p	{0.4, 0.5, 0.6, 0.7}	0.5
L_1	{1000, 2000, 3000, 4000}	4000
L_2	{1000, 2000, 3000, 4000}	1000

Table 2

The comparison results of ParaKplex and the comparison algorithms on five benchmarks. #best denotes the number of best results found by each algorithm on each benchmark and #total denotes the total number of best results found by each algorithm across all benchmarks.

Benchmark	#inst	Heuristic algorithms				Exact algorithms					
		ParaKplex #best	GPULS #best	KLS #best	DCCplex #best	U-MKP #best	DiseMKP #best	KpLeX-Gap #best	kPEX #best	KplexT #best	RelaxPUB-Maplex #best
BHOSLIB	369	361	0	67	150	0	0	0	0	0	0
DIMACS	720	691	127	518	632	293	256	223	265	248	257
Real-world	1251	1247	337	1227	1119	1207	1036	1200	986	1114	1062
DIMACS10	756	747	293	732	694	702	544	537	462	527	499
SNAP	477	477	148	475	419	466	403	445	375	400	383
#total	3573	3523	905	3019	3014	2668	2239	2405	2088	2289	2211

In our experiments, all the algorithms were implemented in C++ and compiled in g++ with '-O3' option. Since GPULS is GPU-based, experiments with GPULS are conducted on NVIDIA GeForce RTX 4090 equipped with 24GB GDDR6X memory. Other algorithms are run on Intel Xeon Gold 6238 CPU @ 2.10GHz CPU with 512GB RAM under Ubuntu 22.04. For each instance, the time limit of the heuristic and exact algorithms is 1000 and 1800 seconds, respectively. Our algorithm sets the number of physical threads to 10 by default. Other heuristic algorithms are executed 10 times with the random seeds from 1 to 10, and exact algorithms are executed only once. We evaluate the performance of the algorithms by considering both the best results achieved by each algorithm and the average time taken to reach those best results. The best overall results are bolded in the table. Our source code and detailed results for all algorithms are available on GitHub¹².

Our algorithm has 11 parameters, with 5 parameters for the group-based search, including β_1 , β_2 , pro , T , and $MaxTry$, other 6 parameters for the local search, including t , L_i , ub_thre , ρ , p , and L . Due to the dense graph structure of classic instances, the edge density between vertices is very high, leading to a large number of candidate variables and a significantly expanded search space during the search process. As a result, classic instances require a higher threshold for the maximum number of non-improving steps compared to large sparse graphs. In our algorithm, we use L_1 to denote the maximum number of non-improving steps for the classic instance and L_2 for the large sparse graph. To tune these parameters, we used the automatic configuration tool irace [50]. Specifically, we randomly selected 10 instances from each of the five benchmarks, and for each instance, we set the nine k values mentioned earlier, resulting in a total of 450 instances for the training set. The tuning process had a budget of 5000 runs, with a time limit of 1000 seconds per run. The final parameter values obtained through this tuning process are listed in Table 1.

8.3. Experimental results

Our experiment compares ParaKplex with the parallel algorithm as well as the sequential heuristic and exact algorithms. Table 2 summarizes the experimental results for ParaKplex and its competitors.

Comparisons with the parallel algorithm. We first compared ParaKplex with the parallel algorithms U-MKP and GPULS. From the experimental results in Table 2, it can be observed that ParaKplex outperforms U-MKP and GPULS in all benchmarks. Specifically, ParaKplex achieves better solutions than U-MKP and GPULS by 23.9% and 73.3%, respectively, across all test instances. Both U-MKP and GPULS failed to find the best result for any instance on the BHOSLIB benchmark, whereas ParaKplex achieved a success

¹² <https://github.com/yiyuanwang1988/Parallel-Kplex>

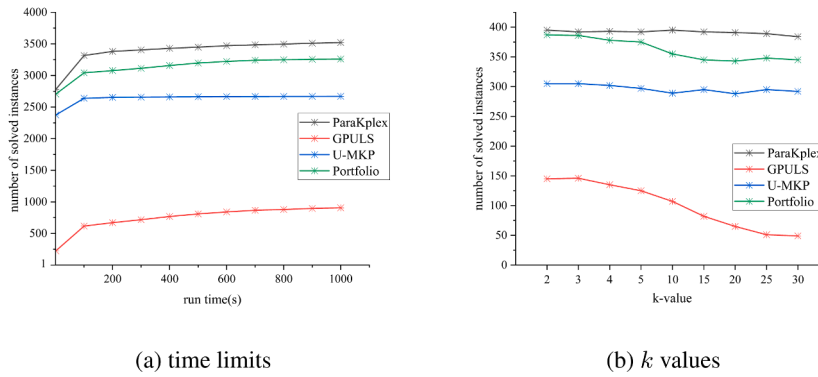


Fig. 2. The number of instances of the best result found by each algorithm under different time limits and k values.

rate of approximately 97.8% on this benchmark. Notably, ParaKplex found the best result for all instances in the SNAP benchmark. This demonstrates ParaKplex's clear superiority over GPULS and U-MKP, both in terms of solving efficiency and its ability to derive high-quality solutions.

Comparisons with the sequential heuristic algorithms. Next, we compare ParaKplex with state-of-the-art sequential heuristic algorithms KLS and DCCplex. The experimental results show that while KLS and DCCplex exhibit competitive advantages, ParaKplex still outperforms these two algorithms across all benchmarks. ParaKplex demonstrated superior performance compared to KLS and DCCplex, identifying the best results in 504 more instances than KLS and 509 more instances than DCCplex. In the two classic benchmarks, ParaKplex outperformed KLS and DCCplex in 474 and 287 instances, respectively, while performing less effectively in only 6 and 17 instances. Across three large sparse graph benchmarks, ParaKplex surpasses KLS and DCCplex in 42 and 251 instances, respectively, with only 6 and 9 instances showing slightly inferior results.

Comparisons with the sequential exact algorithms. We also compare ParaKplex with advanced sequential exact algorithms, including DiseMKP, KpLeX-Gap, kPEX, KplexT and RelaxPUB-Maplex. These five exact algorithms do not perform well on classic instances. Specifically, none of them can obtain the best result for any instance on the BHOSLIB benchmark. Moreover, on the DIMACS benchmark, the number of instances where these algorithms found the best results was less than half of those found by ParaKplex. However, these exact algorithms demonstrate excellent performance on large sparse instances. In particular, KpLeX-Gap can find the best results for more than 87% instances. Moreover, ParaKplex outperforms DiseMKP, KpLeX-Gap, kPEX, KplexT and RelaxPUB-Maplex for 499, 300, 660, 442 and 539 instances on large sparse graphs, while it is defeated for only 0, 5, 5, 2 and 1 instances, respectively. Notably, compared with DiseMKP, KpLeX-Gap, kPEX, KplexT, and RelaxPUB-Maplex, ParaKplex successfully found better results for instances in both classic instances and large sparse graphs, with improvements of 36.2%, 31.6%, 40.9%, 35%, and 37.2%, respectively. This highlights ParaKplex's effectiveness and robustness in solving a wide range of graph instances.

8.3.1. Performance comparison across time limits and k values

In Fig. 2, we evaluate ParaKplex and all comparison algorithms by counting the instances where each finds the best result under different time limits and k values. To establish a more meaningful baseline for the sequential algorithms, we construct a portfolio of all sequential algorithms, denoted as "Portfolio". The result of the Portfolio for each instance is the best solution among all sequence algorithms. In Fig. 2(a), as the runtime increases, the number of instances for which the best result is found also increases for all algorithms. Both ParaKplex and Portfolio have demonstrated outstanding performance. In particular, ParaKplex has achieved the highest count of instances that obtain the best result for every time limit. In Fig. 2(b), all algorithms exhibit relatively stable performance when solving instances for $k=2, 3$, and 4. However, when k is greater than 5, the instances become more challenging to solve, leading to a gradual performance degradation, especially for GPULS. ParaKplex, U-MKP, and Portfolio demonstrate better stability. Although the number of instances where they find the best result decreases as k grows, the fluctuation is minor. Notably, ParaKplex consistently outperforms the other algorithms across different k values.

8.3.2. Runtime efficiency analysis

To further demonstrate the efficiency of our algorithm, we compare the runtime of ParaKplex with that of the heuristic and exact algorithms on instances where both ParaKplex and the corresponding algorithm find the same best results.

We compare the runtime of ParaKplex and the heuristic algorithms on the classic instances and large sparse graphs in Fig. 3(a) and (b), respectively. Compared with GPULS and DCCplex, ParaKplex demonstrates excellent performance in both classic instances and large sparse graphs. For classic instances, ParaKplex spent less time than GPULS and DCCplex on 99.9% and 86.7% of instances to find the same best results, respectively. For large sparse graphs, ParaKplex outperformed GPULS and DCCplex in terms of runtime for 99.8% and 91.7% of instances, respectively. As shown in Fig. 3(a), KLS demonstrates good performance when solving classic instances. Although KLS outperforms ParaKplex in terms of running time for 14.4% of the instances, ParaKplex performs better on large sparse graphs, taking less time than KLS in 83.7% of the instances. Overall, ParaKplex still shows high efficiency compared with heuristic algorithms.

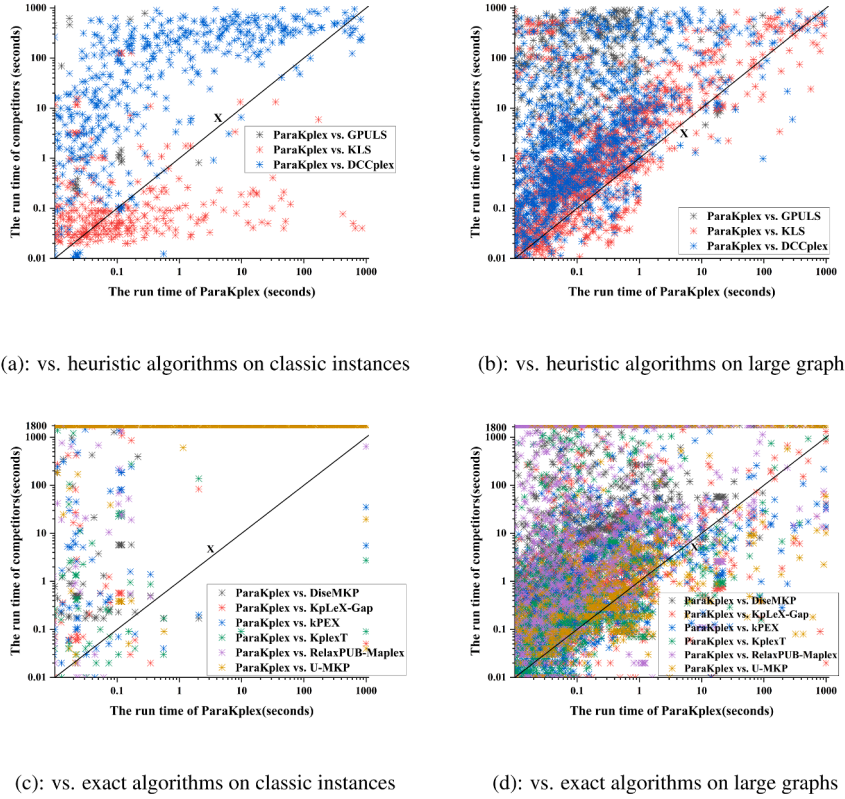


Fig. 3. The runtime comparison of ParaKplex with heuristic and exact algorithms to obtain the best result on all benchmarks.

We compare the runtime of ParaKplex and the exact algorithms on the classic instances and large sparse graphs in Fig. 3(c) and 3(d), respectively. For classic instances, ParaKplex and its competitors can solve most instances within 10 seconds, with the majority of instances concentrated between 0.01 and 1 second. Although ParaKplex has a slightly longer runtime than its competitors in a few instances, it still demonstrates excellent performance in 99.3%, 91.7%, 94.7%, 94%, 94.5% and 80.9% compared with DiseMKP, KpLeX-Gap, kPEX, KplexT, RelaxPUB-Maplex, and U-MKP, respectively. Regarding large sparse graphs, ParaKplex still demonstrates superior performance, solving more than 97% of instances within 100 seconds. Focusing on the comparison between ParaKplex and its competitors, ParaKplex demonstrates higher solution efficiency in over 55% of large sparse graphs.

8.4. The effectiveness of the proposed strategies

In our ablation experiments, we first demonstrated ParaKplex's superiority over state-of-the-art sequential local search algorithms, and subsequently validated the effectiveness of both our proposed parallel and sequential strategies.

8.4.1. The effectiveness of our proposed algorithm

In the following experiments, we first evaluate the performance of our sequential algorithm (Nukplex) [41] against several GDI-based sequential algorithms, and then verify the effectiveness of our proposed parallel algorithm (ParaKplex) by comparing it with Nukplex and these GDI-based algorithms. It should be noted that our proposed ParaKplex is a parallel extension of Nukplex, in which the primary search mode directly invokes the sequential Nukplex algorithm.

Recently, Sun et al. [36] proposed a general group-driven initialization (GDI) method for clique relaxation problems such as k -quasi-clique and k -plex. As for the k -plex problem, they applied this initialization method to the MKPP algorithms FD-TS [37] and DCCplex [5], resulting in the FD-TS_GDI and DCCplex_GDI algorithms. These GDI-based algorithms demonstrate significant superiority over their original versions. Considering the successful application of the GDI method in sequential algorithms, we speculate that integrating GDI into our Nukplex algorithm may further enhance the overall performance. Consequently, we applied the GDI method to Nukplex, denoted as Nukplex_GDI. In our first experiment, we compared Nukplex with Nukplex_GDI, FD-TS_GDI and DCCplex_GDI across all test benchmarks. The comparison results are shown in Table 3.

As shown in Table 3, in the classic benchmarks, Nukplex demonstrates outstanding performance compared to all GDI-based algorithms, with the exception of a tie with DCCplex_GDI in the DIMACS benchmark. However, in the large sparse graphs, Nukplex performs worse than the GDI-based algorithms. On all large sparse graph benchmarks, Nukplex finds fewer best results than both Nukplex_GDI and FD-TS_GDI. Specifically, Nukplex_GDI and FD-TS_GDI outperform Nukplex by 27 and 32 instances, respectively.

Table 3

Comparison results of **Nukplex with GDI-based algorithms** on all benchmarks. #bet and #wor represent the number of instances where Nukplex obtained the better and worse results for each benchmark, respectively.

Benchmark	#inst.	Nukplex vs. Nukplex_GDI #bet(#wor)	Nukplex vs. FD-TS_GDI #bet(#wor)	Nukplex vs. DCCplex_GDI #bet(#wor)
BHOSLIB	369	89 (5)	282 (0)	115 (15)
DIMACS	720	29 (17)	100 (12)	37 (37)
Real-world	1251	2(6)	9(11)	128 (48)
DIMACS10	756	9(26)	12(34)	3(44)
SNAP	477	2(8)	3(11)	1(19)

Table 4

Comparison results of **ParaKplex with Nukplex and GDI-based algorithms** on all benchmarks. #bet and #wor represent the number of instances where ParaKplex obtained the better and worse results for each benchmark, respectively.

Benchmark	#inst.	ParaKplex vs. Nukplex #bet(#wor)	ParaKplex vs. Nukplex_GDI #bet(#wor)	ParaKplex vs. FD-TS_GDI #bet(#wor)	ParaKplex vs. DCCplex_GDI #bet(#wor)
BHOSLIB	369	51 (24)	114 (9)	281 (0)	134 (11)
DIMACS	720	38 (16)	46 (14)	107 (9)	51 (29)
Real-world	1251	12 (3)	8 (4)	8 (2)	132 (48)
DIMACS10	756	45 (1)	34 (4)	35 (8)	14 (4)
SNAP	477	19 (0)	16 (0)	14 (0)	1 (0)

It is worth noting that in the real-world benchmark, Nukplex still maintains a good performance relative to DCCplex_GDI, finding more best results in 80 instances. But in DIMACS10 and SNAP benchmarks, Nukplex performs worse than DCCplex_GDI by 41 and 18 instances, respectively. This indicates that integrating GDI into the algorithm can provide the algorithm with higher-quality initial solutions, thereby accelerating the solving process, which is particularly prominent when dealing with large sparse graphs.

The experiment results in Table 3 show that Nukplex is not as effective as the GDI-based algorithms, especially when solving large sparse graphs. To demonstrate that ParaKplex not only outperforms the Nukplex algorithm, but also surpasses the GDI-based algorithms in terms of performance, in our second experiment, we compared ParaKplex with Nukplex and the GDI-based algorithms. The comparison results are shown in Table 4.

As can be observed from Table 4, ParaKplex outperforms all comparison algorithms across all benchmarks. For classic benchmarks, although Nukplex demonstrates significant advantages, ParaKplex still achieves 27 and 22 more best results than Nukplex on the BHOSLIB and DIMACS benchmarks, respectively. Compared with the GDI-based algorithms, ParaKplex demonstrates excellent performance in classic instances. It outperformed Nukplex_GDI, FD-TS_GDI, and DCCplex_GDI in 137, 379, and 145 instances respectively, representing an improvement ratio of 12.6%, 34.8%, and 13.3%. For large sparse graphs, ParaKplex outperforms Nukplex and all GDI-based algorithms in 76, 58, 57, and 147 instances, showing inferior performance only in 4, 8, 10, and 52 instances. It is worth noting that ParaKplex achieves the best values in all instances in the SNAP benchmark, with no instances performing worse than the comparison algorithms.

In summary, as a parallel extension of Nukplex, ParaKplex employs a suitable group-based initialization method and an effective primary-secondary search method, achieving varying degrees of improvement on the tested benchmarks. Consequently, ParaKplex exhibits superior performance compared to both Nukplex and the GDI-based algorithms.

8.4.2. The effectiveness of parallel strategies

In this subsection, we perform ablation experiments to assess the effectiveness of the parallel strategies employed in our algorithm, specifically the group-based initialization method and the primary-secondary search method. To evaluate the proposed group-based initialization method, we compare ParaKplex with two variants. The first variant is ParaKplex_GDI, which replaces our group-based initialization method with the GDI method proposed by Sun et al. [36]. The second variant is ParaKplex_freqinit, which substitutes our group-based initialization method with the frequency-based initialization method [41]. We further compare ParaKplex with another variant, ParaKplex_nops, which does not use the primary-secondary search method, employing the same search framework as Nukplex. Since these two parallel strategies are designed for large sparse graphs, we evaluate the effectiveness of ParaKplex and the three variants on three large sparse graph benchmarks. The comparison results are shown in Table 5.

The results in Table 5 show that ParaKplex outperforms ParaKplex_nops, ParaKplex_freqinit, and ParaKplex_GDI on 17, 27, and 34 instances, respectively, while it is defeated on 7, 12, and 7 instances. Although the GDI-based algorithm Nukplex_GDI has enhanced the solution quality of Nukplex, when applying GDI to the parallel algorithm, ParaKplex_GDI performs the worst among these three variants. The reason is that the GDI method is a strategy suitable for sequential algorithms, and it lacks a mechanism for sharing useful information among the threads during the search process. Therefore, the algorithm cannot efficiently reduce the problem space and generate high-quality initial solutions. Compared to ParaKplex_freqinit, our group-based initialization method still demonstrates superior performance. Specifically, ParaKplex achieves better results on 15 instances of large sparse graphs. Besides, during each interactive sharing phase, subgroup updates are triggered with an average probability of 5.6%. This indicates that subgroup updates occur infrequently during the search, and the associated computational overhead has only a minor impact on overall efficiency. Fur-

Table 5

Comparison results of **ParaKplex with its variants** on large sparse graphs. #bet and #wor represent the number of instances where ParaKplex obtained the better and worse results for each benchmark, respectively.

Benchmark	#inst.	ParaKplex vs. ParaKplex_nops #bet(#wor)	ParaKplex vs. ParaKplex_freqinit #bet(#wor)	ParaKplex vs. ParaKplex_GDI #bet(#wor)
Real-world	1251	4(1)	8(1)	9(2)
DIMACS10	756	10(6)	16(11)	20(5)
SNAP	477	3(0)	3(0)	5(0)

Table 6

Comparison results of **Nukplex with its variants** on classic and large sparse graph benchmarks. #bet and #wor represent the number of instances where Nukplex obtained the better and worse results for each benchmark, respectively.

Benchmark	#inst.	vs. Nukp_BCC #bet(#wor)	vs. Nukp_DCC #bet(#wor)	vs. Nukp_SCC #bet(#wor)	vs. Nukp_nop #bet(#wor)	vs. Nukp_add #bet(#wor)	vs. Nukp_swap #bet(#wor)	vs. Nukp_str #bet(#wor)
classic	1089	315(22)	202(17)	140(29)	58(27)	61(31)	119(44)	299(43)
large sparse	2484	64(15)	55(17)	31(20)	27(18)	235(7)	106(10)	299(5)

thermore, ParaKplex outperforms ParaKplex_nops on 3, 4, and 3 instances across the Real-world, DIMACS10, and SNAP benchmarks, respectively, demonstrating the effectiveness of the primary-secondary search approach.

8.4.3. The effectiveness of sequential strategies

In this subsection, we present the effectiveness of the proposed sequential strategies in Table 6 which include the RBCC strategy, the perturbation strategy, and the vertex selection strategy.

To validate the effectiveness of our proposed RBCC strategy, we compare our Nukplex algorithm with three alternative algorithms, where Nukp_BCC replaces RBCC with BoundedCC, Nukp_DCC uses DCC instead of RBCC, and Nukp_SCC applies SCC [51] instead of RBCC. Additionally, we compare Nukplex with Nukp_nop, a variant that omits the core-based perturbation strategy. To further assess the vertex selection strategy, we introduce three more variants. Specifically, Nukp_add substitutes our add rule with a random-based method [5,37] in *ConstrAddSet*; Nukp_swap replaces our swap rule with a random-based method in *ConstrSwapSet*; Nukp_str uses a structure-based method [34] instead of our add rule. We compared Nukplex with the above seven variants on all test benchmarks, and the comparison results are summarized in Table 6 for the classic benchmarks and large sparse graph benchmarks.

As shown in Table 6, Nukplex outperforms all RBCC alternative algorithms, especially on the classic benchmarks. Specifically, for the classic benchmarks, Nukplex found better solutions for 293, 185, and 111 more instances than Nukp_BCC, Nukp_DCC, and Nukp_SCC, respectively. On the large sparse graph benchmarks, it outperformed by 49, 38, and 11 more instances, respectively. The results intuitively show that RBCC clearly improves the performance of the Nukplex algorithm. Regarding our proposed perturbation strategy, Nukplex surpasses Nukp_nop in 58 and 27 instances on the classic benchmarks and the large sparse graph benchmarks, respectively, while it is worse than Nukp_nop in 27 and 18 instances. The results show that the core-based perturbation strategy has played a positive role in our algorithm. When comparing Nukplex with the three vertex selection alternative algorithms, Nukp_add, Nukp_swap, and Nukp_str, it was found that Nukplex performed better in achieving the best results. Specifically, on the classic benchmarks, it achieved the best results in 30, 75, and 256 more instances than Nukp_add, Nukp_swap, and Nukp_str, respectively. On the large sparse graph benchmarks, Nukplex demonstrated better results in 228, 96, and 294 more instances, respectively. This demonstrates that the proposed add and swap rules perform better than previous vertex selection rules.

8.5. The robustness of our proposed algorithm

To evaluate our algorithm's robustness, we tested ParaKplex in terms of its scalability and parameter sensitivity to verify its performance stability.

8.5.1. Scalability of ParaKplex

We evaluate the scalability of ParaKplex by analyzing the ratio of algorithm performance to the single thread performance under different numbers of threads. Specifically, we use $Best_N$ to denote the number of best results obtained by the algorithm with N threads, and $Time_N$ to denote the average wall-clock time required to obtain these results, respectively. $Best_{base}$ and $Time_{base}$ denote the number of best results and the average time when using a single thread. We tested each instance with the number of threads in $\{2, 4, 8, 16, 32\}$. Fig. 4 presents the performance of ParaKplex in terms of the ratios $\frac{Best_N}{Best_{base}}$ and $\frac{Time_N}{Time_{base}}$ in classic and large sparse graph benchmarks, respectively.

The red horizontal dotted line in Fig. 4(a) represents the performance of ParaKplex when using a single thread to achieve the best results, and above this line means better performance than the baseline. In the classic and large sparse graph benchmarks in Fig. 4(a), the ratio of $\frac{Best_N}{Best_{base}}$ between N threads and a single thread increases as the number of threads increases. Especially in the classic benchmarks, the solving performance of ParaKplex significantly improves with the increase in the number of threads. When using 32 threads, its performance is enhanced by 1.27 times compared to the baseline. In contrast, the performance of ParaKplex on large sparse graph benchmarks is less sensitive to the number of threads, and the speedup tends to stabilize when the number of

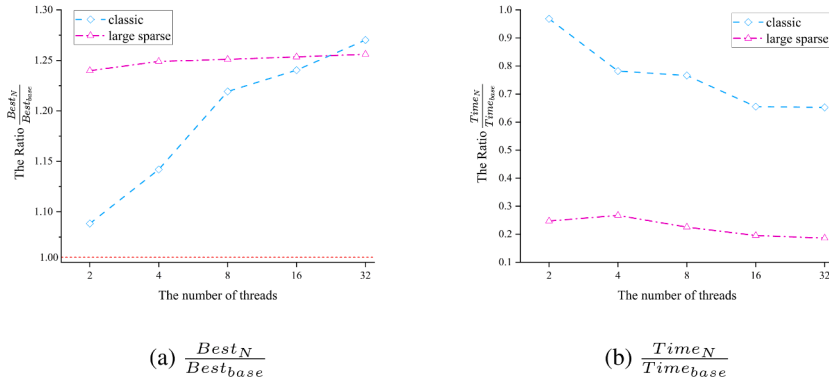


Fig. 4. The ratio of $\frac{Best_N}{Best_{base}}$ and $\frac{Time_N}{Time_{base}}$ when adopting different number of threads.

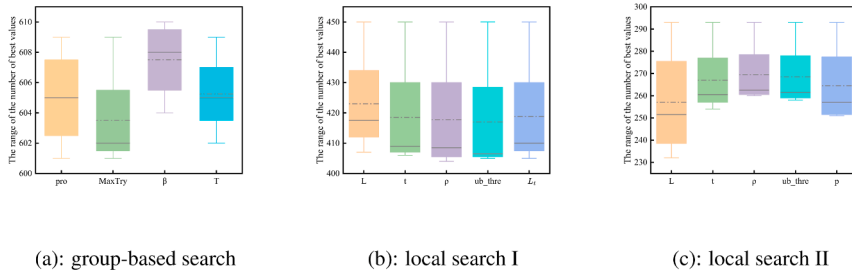


Fig. 5. The parameters used in group-based search and local search were tested.

threads exceeds 8. This is because 98.2% of instances in the large sparse benchmarks have found the best results when ParaKplex runs with 2 threads. Therefore, when the number of threads increases to 32, only 1.3% of instances (from 98.2% to 99.5%) achieve improvements, resulting in an insignificant improvement in overall performance.

As shown in Fig. 4(b), the ratio of $\frac{Time_N}{Time_{base}}$ decreases as the number of threads increases, indicating that the algorithm achieves effective speedup with the increase of threads. In the classic benchmarks, the speedup effect is particularly significant when using 4 and 16 threads. However, in the large sparse graph benchmarks, the speedup ratio improvement of 4 threads is smaller than that of 2 threads. This is because although 4 threads find the best results in more instances, the solution process of these new instances takes a longer time, resulting in a lower speedup than 2 threads.

8.5.2. Parameters sensitivity

In this subsection, we conduct a sensitivity analysis for the parameters listed in Table 1. The sensitivity analysis focuses on the impact of parameters on algorithm robustness. In the experiments, we fix all other parameters and adjust the target parameter one at a time to observe the resulting changes in algorithm performance.

We randomly selected 621 and 324 instances of different sizes from three large sparse graph benchmarks and two classic benchmarks, respectively. For each parameter in Table 1, we counted the number of instances for which the algorithm found the best results under each value, and the results of different values of the same parameter are presented as a box in Fig. 5. The upper and lower whiskers in the box plot represent the maximum and minimum values of the results, respectively. The box shows the middle 50% of the data, ranging from the first quartile to the third quartile. The gray solid line in the box is the median line, indicating the median of the results. The gray dashed line in the plot is the mean, which is the average of all results.

Fig. 5(a) presents the results for the parameters used in group-based search. Since group-based search is primarily designed for large sparse graphs, we conducted tests only on the selected 621 instances. It should be noted that parameters β_1 and β_2 represent the lower and upper bounds of group size, respectively, and they must be set in pairs. Therefore, in Fig. 5(a), we uniformly denote this pair of parameters as β , where β_1 and β_2 are simultaneously set to the i -th value in the range of values during the experiment. From Fig. 5(a), it can be observed that the number of best results obtained by the algorithm fluctuates between 601 and 610 across different parameter values. The box plots demonstrate that the medians under various parameter values are close, ranging approximately from 605 to 608. This indicates that in the group-based search, different parameter values have minimal impact on algorithm performance, demonstrating that the algorithm exhibits strong robustness within the given parameter space.

Fig. 5(b) and (c) present the experimental results of the parameters in the local search on large sparse graphs and classic instances, respectively. In Fig. 5(b), the best results obtained by the algorithm under different parameter values range from 404 to 450, with an average value between 418 and 423. Moreover, the overlapping regions of the boxes for each parameter are relatively large, which means the algorithm's results exhibit good stability under different parameter values. Fig. 5(c) shows that, in the classic instances,

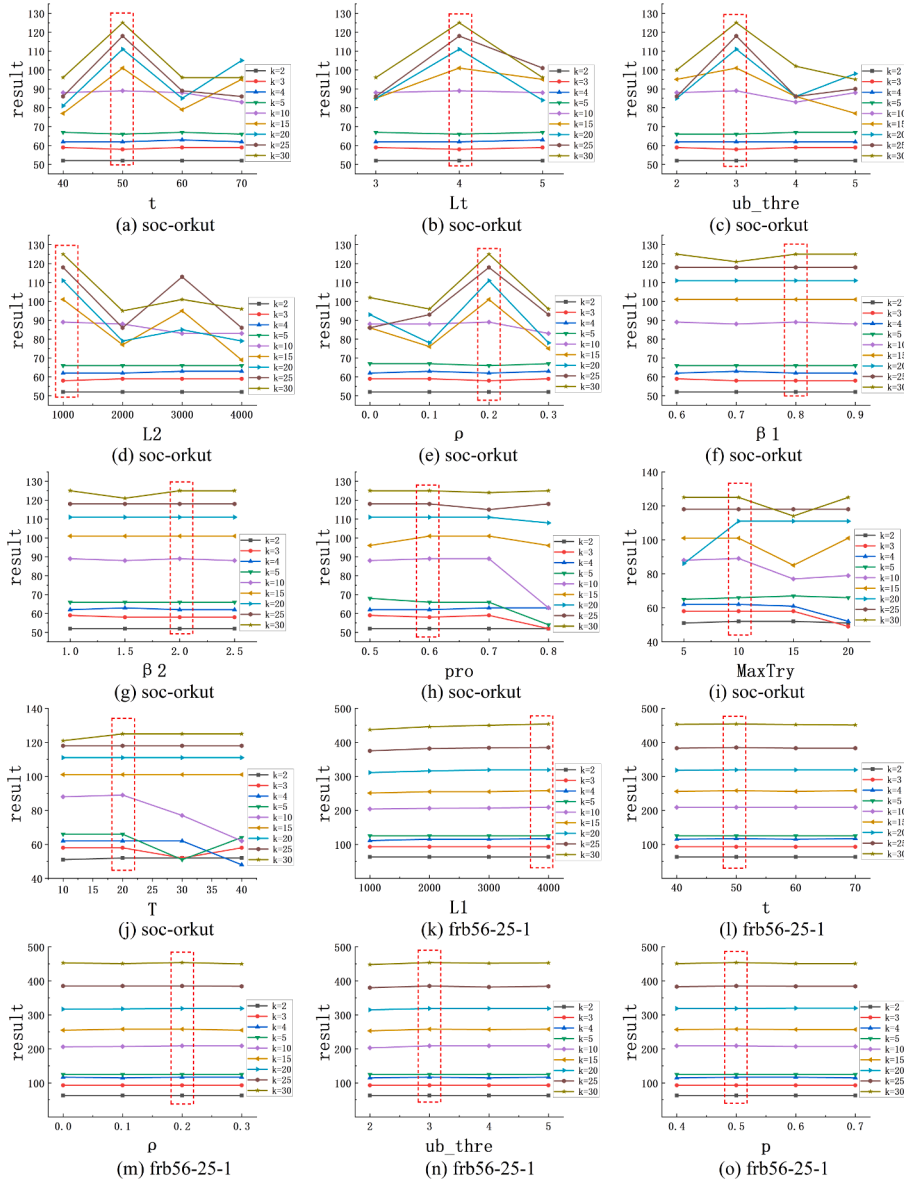


Fig. 6. Analysis of the impact of parameter values on performance.

except for parameter L , whose different values cause a larger fluctuation in results, the best values found by the algorithm under the remaining parameters are all stable within the range of 255 to 292. This suggests that when dealing with classic instances, the algorithm is more significantly impacted by the parameter L . Because classic instances require the non-improvement step L to be sufficiently large, an L value that is too small will directly affect the convergence of the algorithm. However, the algorithm is less sensitive to other parameter values, and its performance remains relatively consistent under different parameter configurations.

To further demonstrate the impact of parameter values on the results, we selected two representative instances (i.e., soc-orkut and frb56-25-1) from the 621 large sparse graphs and the 324 classic instances, respectively. Fig. 6 presents the results for these two instances under different parameter values across various k values, where the red dashed boxes indicate the final value for each parameter. It can be observed that for most parameters, the results exhibit non-monotonic behavior as the parameter values vary. Although the differences in results among different parameter values are relatively small for frb56-25-1, the final parameter values for both soc-orkut and frb56-25-1 are located near the peak where the results are best.

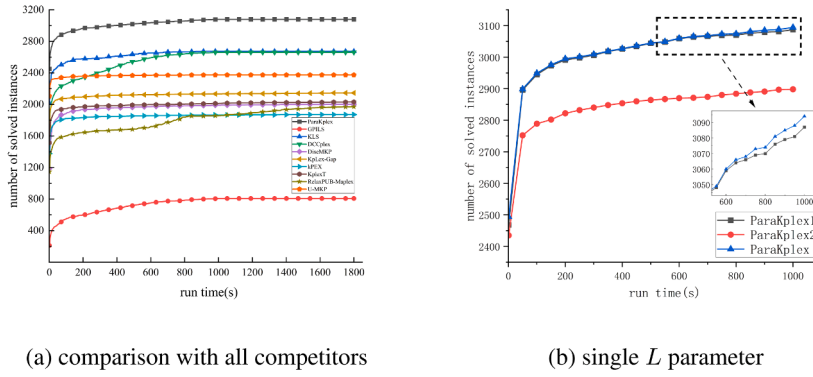


Fig. 7. Cumulative performance curves of each algorithm on the test set.

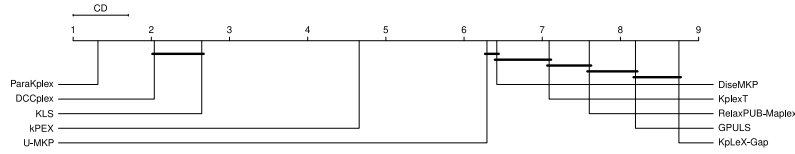


Fig. 8. Critical difference plots about ParaKplex and all competitors on the BHOSLIB benchmark.

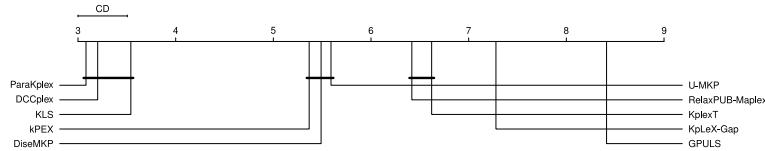


Fig. 9. Critical difference plots about ParaKplex and all competitors on the DIMACS benchmark.

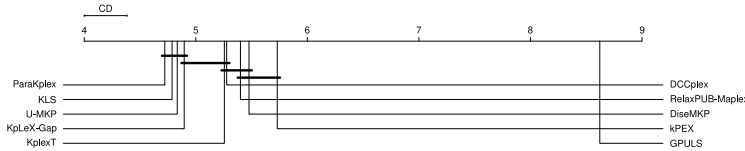


Fig. 10. Critical difference plots about ParaKplex and all competitors on the real-world benchmark.

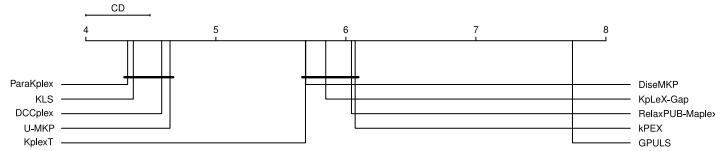


Fig. 11. Critical difference plots about ParaKplex and all competitors on the DIMACS10 benchmark.

8.5.3. Further analysis

To eliminate the influence of the training set used for parameter tuning, we excluded all 450 training instances from the test benchmarks, resulting in a test set of 3123 instances. In this experiment, we compare ParaKplex with all competitors on the test set, with results shown in Fig. 7(a). The performance of ParaKplex with a single L parameter on the test set is presented in Fig. 7(b).

As shown in Fig. 7(a), on the test set, the numbers of instances in which ParaKplex, U-MKP, GPULS, KLS, DCCplex, DiseMKP, KpLeX-Gap, kPEX, KplexT, and RelaxPUB-Maplex obtained the best result are 3094, 2384, 805, 2671, 2668, 2016, 2149, 1868, 2043, and 1759, respectively. ParaKplex achieves the highest solution success rate among all compared algorithms. Notably, when compared with the two parallel algorithms U-MKP and GPULS, ParaKplex solves 29.8% more instances than U-MKP and more than three times as many instances as GPULS. Compared with other algorithms such as KLS, DCCplex, DiseMKP, KpLeX-Gap, kPEX, KplexT, and RelaxPUB-Maplex, ParaKplex achieves improvements in the number of best results found by 15.8%, 15.9%, 53.4%, 43.9%, 65.6%, 51.4%, and 75.8%, respectively. Furthermore, we excluded instances where ParaKplex and the compared algorithms obtained the same results. In terms of solution quality, ParaKplex achieved average improvements of 24.7%, 6.3%, 1.8%, 6.7%, 7.9%, 9.9%,

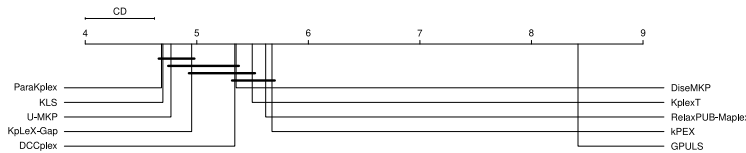


Fig. 12. Critical difference plots about ParaKplex and all competitors on the SNAP benchmark.

9.8%, 7.8%, and 8.2% over GPULS, U-MKP, KLS, DCCplex, DiseMKP, KpLeX-Gap, kPEX, KplexT, and RelaxPUB-Maplex, respectively. Although the average improvement in solution quality of ParaKplex over KLS is only 1.8%, the two algorithms produced the same results on 2697 out of the 3123 test instances. Among the remaining 426 instances where the results differ, ParaKplex outperforms KLS on 423 instances. These results indicate that our algorithm achieves significant improvements over the other algorithms, both in the number of instances where the best result is found and in the solution quality.

In the comparison experiment presented in Section 8.3, ParaKplex uses $L = 1000$ for large sparse graphs and $L = 4000$ for classic instances. It should be noted that only the parameter L is set to different values for different types of instances. Here, we instead apply a single L on the test set. Specifically, ParaKplex1 uses $L = 4000$, while ParaKplex2 uses $L = 1000$. The results are shown in Fig. 7(b). ParaKplex, ParaKplex1, and ParaKplex2 achieve the best result on 3094, 3087, and 2898 instances, respectively. As shown in Fig. 7(b), although the solving performance of ParaKplex and ParaKplex1 is very close, ParaKplex still slightly outperforms ParaKplex1 in terms of the number of solved instances, solving 7 more instances. This indicates that for large sparse graphs, setting $L = 1000$ is more effective than setting $L = 4000$. Although increasing L has a certain impact on algorithm performance, the overall effect is relatively small. However, ParaKplex2 performs significantly worse, solving 6.8% fewer instances than ParaKplex. This suggests that for classic instances with dense structures, decreasing L from 4000 to 1000 may cause the algorithm to insufficiently explore the neighborhood space, thereby limiting its deep search. Overall, the different values of L have a certain impact on the performance of the algorithm, but ParaKplex still exhibits the best performance.

8.6. Critical difference analysis

To ensure a fair comparison, the heuristic algorithms were run 10 times, whereas our algorithm was run once using 10 threads. Since the exact algorithms lack randomness, their results are identical across multiple runs and thus the exact algorithms were run only once. Based on this setup, we utilized the best results achieved by each algorithm within a given instance as the metric to evaluate performance. However, relying solely on the best results may not fully reflect the performance differences. Therefore, we employed non-parametric statistical tests to analyze the differences between ParaKplex and all competitors across five benchmarks.

First, we use the Friedman test [52] with the null-hypothesis that ParaKplex and all its competitors are equivalent in terms of performance. If the null-hypothesis was rejected, we conducted pairwise comparisons using the Nemenyi post-hoc test. All of the above statistical analyses were implemented using the scmamp package [53] in R, which is specifically designed for non-parametric statistical comparisons of multiple algorithms across multiple instances. We employed a critical difference diagram [54] to present the results, as shown in Figs. 8–12.

In the Figures, the top line (i.e., the axis) represents the average rank of the algorithms, with a lower ranking indicating better performance. If there is no significant difference between algorithms at a significance level of 0.05, these algorithms are connected. The results in Fig. 8 indicate that ParaKplex demonstrates statistically significant differences from all competitors on the BHOSLIB benchmark, achieving the best overall performance. Although ParaKplex shows no significant difference from KLS, U-MKP, and DCCplex in Figs. 10–12, it still holds the lowest average rank.

9. Conclusion

In this study, we propose a parallel local search algorithm named ParaKplex for the MKPP. We introduce a group-based initialization method that generates and updates subgroups to construct initial solutions for worker threads. Moreover, we propose a primary-secondary search method with complementary modes to balance global and local exploration. Additionally, our algorithm employs configuration checking, vertex selection, and perturbation strategies to improve search efficiency. Extensive experiments were conducted to evaluate the performance of ParaKplex, and the results demonstrate that it significantly outperforms the comparison algorithms.

In future work, we will further explore the generality of the proposed group-based initialization method and the primary-secondary search framework, and extend them to other relaxed clique problems, such as the k -quasi-clique problem. Although the current study focuses on the maximum k -plex problem, we believe that the underlying design principles, particularly those related to informed initialization and parallel search coordination, are not limited to this specific problem. Instead, they provide a general paradigm that can be adapted to a broader class of combinatorial optimization problems with similar structural characteristics. We hope that this work can serve as a step toward more general and transferable algorithmic frameworks in this research direction.

CRedit authorship contribution statement

Jieyu Wu: Writing – original draft; **Rui Sun:** Writing – original draft; **Yiyuan Wang:** Writing – review & editing; **Minghao Yin:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by National Cryptologic Science Fund of China 2025NCSF02046, [National Natural Science Foundation of China](#) under Grant No. [61806050](#).

Data availability

Data will be made available on request.

Appendix A.

The relevant variables used by each procedure in our algorithm are described in [Table A.1](#).

Table A.1
Descriptions of notations used in this paper.

Variable	Description
Variables in layers and subgroups	
G^{ks}	The set of layers derived from k -shell decomposition.
V_l^{ks}	The l -th layer in G^{ks} .
$d^{ks}(V_l^{ks})$	The vertex degree at the time of removal during the l -th layer decomposition.
w_l^{ks}	The weight of the l -th layer.
d_{avg}^{ks}	The average of the $d^{ks}(V_l^{ks})$ values over all layers V_l^{ks} in G^{ks} .
G^{sg}	The set of subgroups generated in the subgroup generation procedure and updated in the subgroup update procedure.
V_j^{sg}	The j -th subgroup in G^{sg} .
w_j^{sg}	The weight of the j -th subgroup.
$deg_{avg}(V_j^{sg})$	The average degree of the vertices in V_j^{sg} .
Q_{low}	The minimum size of the subgroup in G^{sg} .
Q_{up}	The maximum size of the subgroup in G^{sg} .
Global variable	
S_{gb}	The global best solution.
Local variables in each thread	
S_{in}	The initial solution.
S_{lb}	The local best solution.
$f_{req}(v)$	The frequency of adding and removing v .
$step(v)$	The steps that v remains in the current solution.
$conf(v)$	The configuration of v .
$thred(v)$	The forbidding strength of v .
$age(v)$	The number of steps since v was last removed from the candidate solution.
$score_{esi}$	The current solution information-based scoring function.
$score_{lbs}$	The local best solution information-based scoring function.

References

- [1] A.W. Wolfe, Social network analysis: methods and applications, *Am. Ethnol.* 24 (1) (1997) 219–220.
- [2] R.D. Alba, A graph-theoretic definition of a sociometric clique, *J. Math. Sociol.* 3 (1) (1973) 113–126.
- [3] L.C. Freeman, The sociological concept of "group": an empirical test of two models, *Am. J. Sociol.* 98 (1) (1992) 152–166.
- [4] S.B. Seidman, B.L. Foster, A graph-theoretic generalization of the clique concept, *J. Math. Sociol.* 6 (1) (1978) 139–154.
- [5] P. Chen, H. Wan, S. Cai, J. Li, H. Chen, Local search with dynamic-threshold configuration checking and incremental neighborhood updating for maximum k -plex problem, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 2343–2350.
- [6] J. Gao, Z. Xu, R. Li, M. Yin, An exact algorithm with new upper bounds for the maximum k -defective clique problem in massive sparse graphs, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022, pp. 10174–10183.
- [7] J. Chen, S. Cai, S. Pan, Y. Wang, Q. Lin, M. Zhao, M. Yin, NuQClq: an effective local search algorithm for maximum quasi-clique problem, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, pp. 12258–12266.

- [8] H. Zhai, M. Haraguchi, Y. Okubo, E. Tomita, A fast and complete algorithm for enumerating pseudo-cliques in large graphs, *Int. J. Data Sci. Anal.* 2 (3) (2016) 145–158.
- [9] L. Chen, Q. Zhou, D. Zhao, k-plex-based community detection with graph neural networks, *Inf. Sci.* 689 (2025) 121509.
- [10] A.L. Kavanaugh, D.D. Reese, J.M. Carroll, M.B. Rosson, Weak ties in networked communities, *Inf. Soc.* 21 (2) (2005) 119–131.
- [11] V.E. Krebs, Mapping networks of terrorist cells, *Connections* 24 (3) (2002) 43–52.
- [12] L. Chang, K. Yao, Maximum k-plex computation: theory and practice, *Proc. ACM Manag. Data* 2 (1) (2024) 1–26.
- [13] S. Gao, K. Yu, S. Liu, C. Long, Maximum k-plex search: an alternated reduction-and-bound method, *Proc. VLDB Endow.* 18 (2025) 363–376.
- [14] Y. Zhou, S. Hu, M. Xiao, Z.-H. Fu, Improving maximum k-plex solver via second-order reduction and graph color bounding, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, pp. 12453–12460.
- [15] B. Balasundaram, S. Butenko, I.V. Hicks, Clique relaxations in social network analysis: the maximum k-plex problem, *Oper. Res.* 59 (1) (2011) 133–142.
- [16] J. Gao, J. Chen, M. Yin, R. Chen, Y. Wang, An exact algorithm for maximum k-plexes in massive graphs, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2018, pp. 1449–1455.
- [17] S.H. Matsugu S, Efficient maximum k-plex search via selective branch-and-Bound, in: *International Conference on Information Integration and Web Intelligence*, 2023, pp. 348–357.
- [18] B. McClosky, I.V. Hicks, et al., Combinatorial algorithms for the maximum k-plex problem, *J. Comb. Optim.* 23 (1) (2012) 29.
- [19] H. Moser, R. Niedermeier, M. Sorge, Exact combinatorial algorithms and experiments for finding maximum k-plexes, *J. Comb. Optim.* 24 (2012) 347–373.
- [20] Y. Okubo, M. Matsudaira, M. Haraguchi, Detecting maximum k-plex with iterative proper l-Plex search, in: *International Conference on Discovery Science*, 2014, pp. 240–251.
- [21] K. Wu, J. Gao, R. Chen, X. Cui, Vertex selection heuristics in branch-and-bound algorithms for the maximum k-plex problem, *Int. J. Artif. Intell. Tools* 28 (05) (2019) 1950015.
- [22] M. Xiao, W. Lin, Y. Dai, Y. Zeng, A fast algorithm to compute maximum k-plexes in social network analysis, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017, pp. 919–925.
- [23] L. Chang, M. Xu, D. Strash, Efficient maximum k-plex computation over large sparse graphs, *Proc. VLDB Endow.* 16 (2) (2022) 127–139.
- [24] H. Jiang, D. Zhu, Z. Xie, S. Yao, Z.-H. Fu, A new upper bound based on vertex partitioning for the maximum K-plex problem, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2021, pp. 1689–1696.
- [25] H. Jiang, F. Xu, Z. Zheng, B. Wang, W. Zhou, A refined upper bound and inprocessing for the maximum K-plex problem, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2023, pp. 5613–5621.
- [26] Z. Wang, Y. Zhou, C. Luo, M. Xiao, A fast maximum k-plex algorithm parameterized by the degeneracy gap, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2023, pp. 5648–5656.
- [27] J. Zheng, M. Jin, K. He, Exact algorithms with new upper bounds for the maximum k-plex problem, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2025.
- [28] A. Ahmad, D. Yan, X. Chen, L. Yuan, Q. Zhang, S. Adhikari, Maximum k-plex finding: choices of pruning techniques matter!, *Proc. VLDB Endow.* 18 (9) (2025) 2928–2940.
- [29] S. Cai, J. Lin, C. Luo, Finding a small vertex cover in massive sparse graphs: construct, local search, and preprocess, *J. Artif. Intell. Res.* 59 (2017) 463–494.
- [30] P. Chen, H. Wan, S. Cai, W. Luo, J. Li, Combining reinforcement learning and configuration checking for maximum k-plex problem, (2019). arXiv:1906.02578
- [31] A. Conte, D. Firmani, C. Mordente, M. Patrignani, R. Torlone, Cliques are too strict for representing communities: finding large k-plexes in real networks, in: *Proceedings of the 26th Italian Symposium on Advanced Database Systems*, vol. 2161, 2018, pp1.
- [32] S.-Y. Hsieh, S.-S. Kao, Y.-S. Lin, A swap-based heuristic algorithm for the maximum k-Plex problem, *IEEE Access* 7 (2019) 110267–110278.
- [33] Z. Miao, B. Balasundaram, Approaches for finding cohesive subgroups in large-scale social networks via maximum k-plex detection, *Networks* 69 (4) (2017) 388–407.
- [34] W. Pullan, Local search for the maximum k-plex problem, *J. Heuristics* 27 (3) (2021) 303–324.
- [35] B. Nogueira, R.G.S. Pinheiro, A GPU based local search algorithm for the unweighted and weighted maximum s-plex problems, *Ann. Oper. Res.* 284 (1) (2020) 367–400.
- [36] R. Sun, Y. Wang, M. Yin, Improving local search algorithms for clique relaxation problems via group driven initialization, *Front. Comput. Sci.* 19 (6) (2025) 1–12.
- [37] Y. Zhou, J.-K. Hao, Frequency-driven tabu search for the maximum s-plex problem, *Comput. Oper. Res.* 86 (2017) 65–78.
- [38] S.B. Seidman, Network structure and minimum degree, *Soc. Netw.* 5 (3) (1983) 269–287.
- [39] S. Cai, K. Su, A. Sattar, Local search with edge weighting and configuration checking heuristics for minimum vertex cover, *Artif. Intell.* 175 (9–10) (2011) 1672–1696.
- [40] F. Glover, Tabu search—part I, *ORSA J. Comput.* 1 (3) (1989) 190–206.
- [41] R. Sun, Y. Wang, H.L. Shimao Wang, X. Li, M. Yin, Nukplex: an efficient local search algorithm for maximum K-plex problem, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2024, pp. 7029–7037.
- [42] J. Shirako, N. Vrvilo, E.G. Mercer, V. Sarkar, Design, verification and applications of a new read-write lock algorithm, in: *Proceedings of the Twenty-Fourth Annual ACM Symposium on Parallelism in Algorithms and Architectures*, 2012, pp. 48–57.
- [43] A. Lipowski, D. Lipowska, Roulette-wheel selection via stochastic acceptance, *Phys. A: Stat. Mech. Appl.* 391 (6) (2012) 2193–2196.
- [44] J. Chen, Y. Wang, S. Cai, M. Yin, Y. Zhou, J. Wu, NukCP: an improved local search algorithm for maximum k-club problem, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022, pp. 10146–10155.
- [45] Q. Wu, J.-K. Hao, A review on algorithms for maximum clique problems, *Eur. J. Oper. Res.* 242 (3) (2015) 693–709.
- [46] K. Xu, F. Boussemart, F. Hemery, C. Lecoutre, Random constraint satisfaction: easy generation of hard (satisfiable) instances, *Artif. Intell.* 171 (8–9) (2007) 514–534.
- [47] D.S. Johnson, M.A. Trick, Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge, October 11–13, 1993, Vol. 26, American Mathematical Soc., 1996.
- [48] R. Rossi, N. Ahmed, The network data repository with interactive graph analytics and visualization, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015, pp. 4292–4293.
- [49] J. Leskovec, K.J. Lang, A. Dasgupta, M.W. Mahoney, Community structure in large networks: natural cluster sizes and the absence of large well-defined clusters, *Internet Math.* 6 (1) (2009) 29–123.
- [50] M. López-Ibáñez, J. Dubois-Lacoste, L.P. Cáceres, M. Birattari, T. Stützle, The irace package: iterated racing for automatic algorithm configuration, *Oper. Res. Perspect.* 3 (2016) 43–58.
- [51] Y. Wang, S. Cai, J. Chen, M. Yin, SCCWalk: an efficient local search algorithm and its improvements for maximum weight clique problem, *Artif. Intell.* 280 (2020) 103230.
- [52] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, *J. Am. Stat. Assoc.* 32 (200) (1937) 675–701.
- [53] B. Calvo, G. Santafé, scmamp: statistical comparison of multiple algorithms in multiple problems, *R J.* 8 (1) (2016) 248–256.
- [54] S. Garcia, F. Herrera, An extension on “statistical comparisons of classifiers over multiple data sets” for all pairwise comparisons, *J. Mach. Learn. Res.* 9 (12) (2008) 2677–2694.